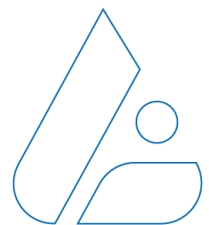


LANACCESS

Discover the power of video.



NXT white paper

Sistema de Gestión de Configuración

1. Visión General

El **Sistema de Gestión de Configuración** ha sido diseñado para proporcionar una infraestructura unificada para la gestión, validación y persistencia de la configuración del grabador NXT.

Su arquitectura se aleja del modelo tradicional de "ficheros de configuración dispersos" para adoptar un enfoque de **modelo de datos centralizado**, accesible a través de múltiples interfaces pero gobernado por una única lógica de negocio.

Principios Fundamentales del Diseño

1. Fuente Única de Verdad (Single Source of Truth):

- La definición de qué es una propiedad válida (tipo, rango, formato, valor por defecto) reside exclusivamente en el registro de la propiedad.
- Ni la Interfaz Web (Angular) ni la CLI (Consola) contienen lógica de validación duplicada. Ambas consultan dinámicamente al backend para saber "qué se puede configurar y cómo", garantizando que la interfaz siempre esté sincronizada con las capacidades reales del firmware.

2. Validación Centralizada y Robusta:

- Ningún cambio de configuración se aplica sin pasar antes por el **Servicio de Validación**.
- Este servicio actúa como un "portero" estricto que rechaza datos mal formados, fuera de rango o que violen reglas de integridad, protegiendo al sistema de estados inconsistentes.

3. Abstracción de Datos (Árbol Virtual):

- Aunque los datos se almacenan de forma eficiente como pares clave-valor planos, el sistema los proyecta hacia el exterior como un **árbol jerárquico JSON** rico y navegable (`config.device.alarms...`).
- Esto permite a los clientes (Web, Scripts) trabajar con objetos completos y estructuras lógicas en lugar de listas interminables de variables.

4. Consistencia Transaccional (Atomicidad):

- Las operaciones de modificación (`POST`, `PUT`, `PATCH` y `DELETE`) están diseñadas para ser atómicas: se valida el conjunto completo de cambios y, solo si todo es correcto, se aplica en bloque. O todo se guarda, o nada cambia.

5. Seguridad por Diseño:

- El sistema integra mecanismos nativos para la protección de datos sensibles (protección contraseñas) y control de concurrencia (sistema de *Locking*) para evitar conflictos entre múltiples administradores.

Componentes Clave

El sistema se orquesta mediante la interacción de tres actores principales sobre el bus del sistema (D-Bus):

- **Microservicios Propietarios (Registradores):** Módulos como `core-legacy` que, al arrancar, "registran" sus propiedades en el sistema central y/u otros como `network-manager` que delegan en la propia imagen yocto el haber dejado en la carpeta de registro un fichero con las propiedades definidas.
- **Servicio de Validación (El Cerebro):** Mantiene el árbol del esquema en memoria, responde a consultas de estructura (`get_schema`) y valida peticiones (`validate`).
- **Interfaces de Acceso (REST & CLI):** Pasarelas que traducen las peticiones de usuario (HTTP o Texto) a comandos internos del sistema, gestionando la sesión y la seguridad.

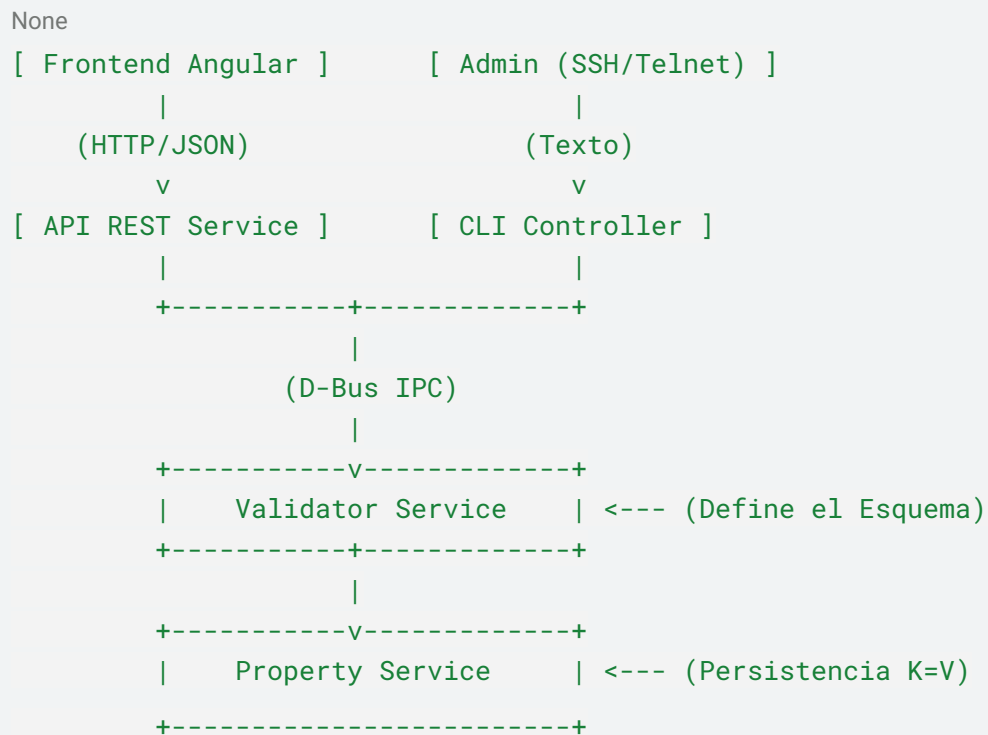
Esta arquitectura modular permite que el sistema crezca indefinidamente: añadir una nueva funcionalidad al equipo solo requiere que el nuevo microservicio registre sus propiedades; la Web y la CLI las "descubrirán" y gestionarán automáticamente sin necesidad de reprogramarlas.

2. Arquitectura de Componentes

El sistema se divide en cuatro capas principales:

1. **Capa de Almacenamiento (system properties):** Gestiona la persistencia de datos clave-valor.
2. **Capa de Lógica y Esquema (system properties validator):** Define qué es válido, genera esquemas y valida cambios.
3. **Capa de Interfaz (API REST & CLI):** Expone la funcionalidad al exterior.
4. **Capa de Recursos (Resources API):** Informa sobre las capacidades físicas o lógicas disponibles.

Diagrama Conceptual



3. Modelo de Datos

El sistema utiliza un modelo de datos híbrido que combina la eficiencia del almacenamiento plano con la expresividad de una estructura jerárquica orientada a objetos.

3.1. Almacenamiento vs. Proyección

- **Almacenamiento Interno (Plano):**

En el nivel más bajo (persistencia y D-Bus)

- *Ejemplo:* `"config.device.main.name" = "NVR-01"`
- *Ventaja:* Acceso $O(1)$, simplicidad de serialización, fácil de replicar o hacer copias de seguridad.

- **Proyección Virtual (Jerárquica):**

Para las capas de aplicación (API REST, CLI, Frontend), el sistema reconstruye en tiempo real un árbol de objetos JSON/YAML

```
{
  "config": {
    "device": {
      "main": {
        "name": "NVR-01"
      }
    }
  }
}
```

- *Ventaja:* Navegación intuitiva, agrupación lógica de parámetros y compatibilidad con frameworks modernos como Angular.

3.2 Sistema de Tipos Extendido

Más allá de los tipos primitivos básicos, el sistema implementa **Tipos Semánticos** que encapsulan reglas de validación y comportamiento de interfaz de usuario.

Tipo	Descripción	Ejemplo de Valor	Validación Automática
<code>string</code>	Texto libre.	<code>"Sala 1"</code>	Longitud máxima (<code>maxLength</code>).

numeric	Enteros (con signo/sin signo).	8080	Rango numérico (<code>minValue</code> , <code>maxValue</code>).
decimal	decimales (con signo/sin signo)	23.07	Rango numérico (<code>minValue</code> , <code>maxValue</code>) y número de decimales (<code>numDigits</code>)
boolean	Lógica binaria.	"true" / "false"	Restringido a estos dos valores.
list	Enumeración cerrada (Select).	"spanish"	El valor debe existir en <code>allowedValues</code> .
flag	Máscara de bits (Multi-select).	"motion+input"	Cada token debe ser válido; combinación permitida.
ipaddr	Dirección IPv4.	"192.168.1.10"	Formato A.B.C.D y rangos de octetos (0-255).
ipmask	Máscara de subred.	"255.255.255.0" "	Formato IP y contigüidad de bits a 1.
password	Credencial segura.	<code>__reserved_pwd</code> _	Write-only: Lectura enmascarada. Validación de complejidad (Niveles 0-4).

Tipos de Recurso (`resource_flag`, `resource_list`)

Estos son tipos avanzados diseñados para hardware o entidades del sistema.

- **Concepto:** En lugar de una lista estática de strings, la lista de valores válidos se genera dinámicamente a partir de la definición del hardware (ej. número de entradas físicas).
- **Formato:** Utilizan prefijos estándar (`in01`, `cam05`).
- **Validación:** El sistema valida que el índice esté dentro del rango real disponible en el dispositivo.

3.3. Plantillas y Colecciones (Arrays)

El sistema maneja colecciones de objetos repetitivos (como 16 entradas de alarma o 4 streams de vídeo) mediante un potente sistema de **Plantillas de Rutas**.

1. **Definición (Esquema):** Se utiliza un **Placeholder** entre llaves para indicar una sección variable.
 - Path: `config.video.camera.{id}.stream.{streamId}.bitrate`
 - Esto define una estructura anidada de profundidad arbitraria.

2. **Instanciación (Datos):** Los datos reales utilizan índices numéricos formateados.
 - Clave: `config.video.camera.000.stream.01.bitrate`

3. **Reglas de Índice (** Cada placeholder está gobernado por una regla estricta definida en el registro:
 - **Rango:** `[min, max]` (ej. 0 a 127 cámaras).
 - **Formato (Padding):** Número de dígitos (ej. `padding: 2` fuerza `"00"`, `"01"`).
 - **Tipo de Colección:**
 - **(Estática):** Recursos hardware (ej. Entradas). Siempre existen todas las instancias del rango. No se pueden crear ni borrar.
 - **(Dinámica):** Entidades lógicas (ej. Usuarios, Alarmas). Empiezan vacías. Se crean (`POST`) y borran (`DELETE`) bajo demanda.

4. El Servicio de Validación (Validator Service)

El **Validator Service** es un componente central y "cerebro" del sistema de configuración. Su responsabilidad principal es mantener la integridad del modelo de datos, actuando como la autoridad única sobre qué configuración es válida y cómo debe estructurarse.

El Validator Service no guarda los valores de configuración (eso es tarea del *System properties Service*), sino que guarda las **definiciones** (metadatos) de cada parámetro.

Funciones Principales

1. **Repositorio de Esquemas:** Mantiene en memoria un árbol completo que representa la estructura ideal de la configuración. Sabe que `config.device.name` es un string de 21 caracteres, o que `config.network.port` es un número entre 1 y 65535.
2. **Motor de Validación:** Proporciona una interfaz RPC (D-Bus) para validar cualquier clave o valor antes de que se escriba en el sistema.
3. **Introspección:** Permite a otros servicios (y al frontend) consultar la estructura del sistema ("¿Qué parámetros tiene una cámara?") y sus reglas, exportando esta información en JSON o YAML.

Registro de Propiedades (Bootstrap)

El sistema es dinámico. Al arrancar, el Validator Service comienza vacío y/o con los ficheros de definición estáticos que existan en la imagen. Son los propios microservicios funcionales (core-legacy) los que, al iniciarse, se conectan al Validator y "registran" sus propiedades.

- **Descentralización de la Definición:** Cada módulo es dueño de su configuración. El módulo de vídeo define qué es un stream válido.
- **Centralización de la Validación:** Una vez registrado, el Validator asume el control de hacer cumplir esas reglas globalmente.

Mecanismos de Registro

Para facilitar esta tarea a los desarrolladores se permiten dos maneras de registrar.

O bien creando ficheros yaml y ubicándolos en la imagen en la carpeta:
`/apps/la-system-properties-validator/conf`.

O bien programáticamente en C++. Se ha creado la librería *header-only* `sysprop_client.hpp`, que ofrece una API de alto nivel, segura y declarativa.

Validación en Tiempo Real

Cuando un cliente (API REST o CLI) intenta modificar una configuración, el Validator Service ejecuta una verificación exhaustiva:

1. **Existencia:** ¿La clave existe en el esquema registrado?
2. **Tipo:** ¿El valor coincide con el tipo esperado (número, IP, booleano)?
3. **Restricciones:** ¿El número está dentro del rango? ¿El string cumple la longitud máxima? ¿La opción está en la lista de permitidos? ¿La contraseña cumple la política de complejidad?
4. **Recursos:** Para tipos `resource_*`, ¿el índice solicitado corresponde a un recurso hardware real?

Sólo si todas las comprobaciones pasan, el sistema permite proceder con la escritura, garantizando así la estabilidad del dispositivo.

4.1. Registro de Propiedades

Como hemos mencionado existen dos formas de registrar las propiedades: mediante ficheros json o programáticamente mediante llamadas dbus.

4.2 Guía de definición de Esquemas de Configuración (YAML)

El sistema carga automáticamente todos los archivos `.yaml` ubicados en el directorio de `/apps/la-system-properties-validator/conf` al inicio. Estos archivos definen la estructura, tipos, validaciones y valores por defecto que serán expuestos a través de la API REST y la consola de administración.

4.2.1 Estructura Básica

El archivo YAML debe reflejar la jerarquía exacta de las propiedades en el sistema (ej. `config.device.users`).

Cada nodo del árbol puede ser:

1. **Un Contenedor:** Un nodo que agrupa otros nodos (ej. `device`).
2. **Una Propiedad:** Un nodo final que tiene un valor (ej. `ip`). Se identifica porque tiene el campo `_type`.
3. **Una Plantilla (Array):** Un nodo que define una colección de elementos repetitivos (ej. `cameras`). Se identifica por el campo `_rule`.

Metadatos

Todos los campos de configuración del propio esquema comienzan con un guión bajo `_` para distinguirlos de las propiedades de datos.

Estos son los atributos comunes a todos los tipos de parámetros.

- `_type`: (Obligatorio) El tipo de dato (`string`, `numeric`, `boolean`, etc.).
- `_description`: (Opcional) Texto de ayuda para el usuario.
- `_default`: (Opcional) Valor por defecto.
- `_rebootRequired`: (Opcional) `true` si cambiar este valor requiere reinicio. Default `false`.
- `_forceSave`: (Opcional) `true` si se guarda la propiedad aunque sea igual al valor de defecto. Default `false`.

4.2.2 Tipos de datos soportados

4.2.2.1. String (`string`)

Cadena de texto alfanumérica.

```
hostname:
  _type: string
  _description: "Nombre del dispositivo"
  _default: "NVR-Default"
  _maxLength: 64
```

Para el caso de valores por defecto de recursos (nombres de defecto de cámaras, entradas, salidas), el valor por defecto puede aceptar placeholders que se instanciarán dinámicamente. Por ejemplo "Input {id}" o "Remote Input {dev}{id}" o si se prefiere "Output {}", "Remote Output {}-{}".

4.2.2.2. Numérico (`numeric`)

Valor entero.

```
port:
  _type: numeric
  _description: "Puerto de servicio"
  _default: 8080
  _minValue: 1
  _maxValue: 65535
```

4.2.2.3. Decimal (`decimal`)

Valor numérico de punto flotante (con decimales). Permite especificar la precisión deseada.

Parámetros:

- `_minValue` (Opcional): Valor mínimo permitido (puede tener decimales).
- `_maxValue` (Opcional): Valor máximo permitido (puede tener decimales).
- `_numDigits` (Opcional): Número de dígitos decimales para el formateo y validación (por defecto 2).

Ejemplo:

```
temperature_threshold:
  _type: decimal
  _description: "Umbral de temperatura crítica"
  _default: 37.5
  _minValue: -10.0
  _maxValue: 80.0
  _numDigits: 1
  _rebootRequired: false
```

```
_mustSave: true
```

4.2.2.3. Booleano (boolean)

Valor verdadero/falso.

```
enabled:
  _type: boolean
  _description: "Habilitar servicio"
  _default: true
```

4.2.2.4. Lista (list)

Selección única entre una lista fija de opciones.

```
protocol:
  _type: list
  _description: "Protocolo de transporte"
  _default: "tcp"
  _allowedValues:
    - "tcp"
    - "udp"
    - "http"
```

4.2.2.5. Flag (flag)

Selección múltiple (combinación de bits). Los valores se guardan separados por + (ej. "read+write").

```
permissions:

  _type: flag
  _description: "Permisos de usuario"
  _allowedValues:
    - "read"
    - "write"
    - "execute"
```

4.2.2.6. Dirección IP (ipaddr)

Valida formato IPv4.

```
address:
  _type: ipaddr
  _default: "192.168.1.10"
```

4.2.2.7. Password (password)

Cadena protegida. Nunca se devuelve el valor real en las lecturas (se devuelve `__reserved_pwd__`).

```
secret:
  _type: password
  _maxLength: 32
  _policyLevel: 2
```

4.2.2.8. Recursos (resource_list, resource_flag)

Listas o flags cuyos valores permitidos dependen de recursos del sistema (ej. entradas físicas).

```
sendCamera:
  _type: resource_list
  _description: "Cámara asociada"
  _format: "cam%03d"
  _size: 150
```

4.2.4. Definición de Arrays y Tablas (Plantillas)

Para definir una lista de objetos (como una lista de usuarios o alarmas), se utiliza un nodo especial con un nombre entre llaves (ej. "{id}") y un bloque `_rule`.

Parámetros de `_rule`

- `range`: [min, max]. El rango de índices válidos (inclusive).
- `padding`: Número de dígitos para el índice (ej. 2 genera "00", "01").
- `fixed`:
 - `true`: **Colección Estática**. Los elementos siempre existen (ej. entradas físicas 1-16). No se pueden borrar ni crear.
 - `false`: **Colección Dinámica**. Los elementos se crean y borran bajo demanda (ej. usuarios, alarmas).

Ejemplo: Tabla Dinámica de Usuarios

```
config:
  users:
    # Definición de la plantilla. El nombre entre {} es el placeholder.
    "{userId}":
      _rule:
        range: [0, 9]    # Máximo 10 usuarios (del 0 al 9)
        padding: 2       # Formato: 00, 01...
        fixed: false     # Se pueden crear y borrar

    # Propiedades de CADA usuario
    name:
      _type: string
      _description: "Nombre de usuario"

    level:
      _type: numeric
      _default: 1
```

Ejemplo: Tabla Fija de Entradas

```
config:
  io:
    inputs:
      "{inputId}":
        _rule:
          range: [0, 7]  # 8 Entradas fijas
          padding: 2
          fixed: true     # Siempre existen, no se borran
```

```
enabled:
  _type: boolean
  _default: false
```

4.2.5. Valores por Defecto Avanzados

Además del valor `_default` estático, el sistema soporta dos mecanismos para definir valores iniciales dinámicos que se adaptan al contexto del despliegue o de la instancia.

4.5.1. Defaults por Personalización (`_customDefaults`)

Permite definir valores por defecto alternativos que se activan automáticamente según la personalización del equipo (ej. "Banco A", "Transporte B").

- Uso: Definir un mapa clave-valor donde la clave es el identificador de la personalización (string) y el valor es el default a aplicar.
- Prioridad: Si la personalización activa del sistema coincide con una clave del mapa, este valor tiene prioridad sobre `_default`.

```
http-port:
  _type: numeric
  _default: 80          # Valor estándar
  _customDefaults:
    bbva: 2080          # Valor si la custom es 'bbva'
    caixa: 8080         # Valor si la custom es 'caixa'
```

4.5.2. Defaults Calculados (`_defaultFunc`)

Permite delegar el cálculo del valor por defecto a una función C++ registrada en el servidor. Esto es ideal para valores que dependen del índice de la instancia (ej. "Cámara 1", "Cámara 2") o de lógica compleja.

- Uso: Especificar el nombre de la función registrada.
- Funcionamiento: El servidor ejecutará la función pasando el contexto de índices actual (ej. `[0]` para la primera cámara) y usará el resultado como valor por defecto.
- Prioridad: Tiene prioridad sobre `_default` pero es anulado por `_customDefaults` si existe una coincidencia.

```
camera-name:
  _type: string
  _defaultFunc: "generate_camera_name" # Generará "Camera 1", "Camera 2"...
```

Lista de funciones de default registradas:

- `on_default_set_id`: devuelve el último índice más uno. Se utiliza para calcular el logical id de las cámaras.
- `on_default_recorder_condition_type`: devuelve "no" (no se graba) para todos los codecs excepto el primero que devuelve "permanent".
- `on_default_set_camera_name`: devuelve el nombre de defecto de la cámara según idioma configurado.
- `on_default_set_camera_input_name`: devuelve el nombre de defecto de la entrada digital de la cámara según idioma configurado.
- `on_default_set_camera_output_name`: devuelve el nombre de defecto de la salida digital de la cámara según idioma configurado.
- `on_default_set_camera_analytics_input_name`: devuelve el nombre de defecto de la entrada de analítica de la cámara según idioma configurado.
- `on_default_set_camera_audio_detection_input_name`: devuelve el nombre de defecto de la entrada de detección de audio de la cámara según idioma configurado.

4.5.3 Jerarquía de Resolución de Defaults

Al determinar el valor inicial de una propiedad, el sistema sigue este orden estricto:

1. Custom Default: Si existe una entrada para la customización actual.
2. Default Function: Si existe una función registrada.
3. Static Default: El valor de `_default`.
4. Implicit Default: 0 para números, "" para strings, `false` para booleanos.

4.2.6 Ejemplo Completo

config:

modules:

mqtt-bridge:

mqtt-broker:

=====

1. CONFIGURACIÓN DEL SERVIDOR (LISTENER EXTERNO)

=====

server:

enabled:

_type: boolean

_description: "Habilitar la integración MQTT hacia el exterior"

_default: false

_rebootRequired: true

port:

_type: numeric

_description: "Puerto de escucha para clientes externos"

_default: 8883

_minValue: 1

_maxValue: 65535

_rebootRequired: true

protocol:

_type: list

_description: "Protocolo de transporte (TCP crudo o WebSockets)"

_default: "mqtt"

_allowedValues:

- "mqtt"

- "websockets"

_rebootRequired: true

tls_enabled:

_type: boolean

_description: "Requerir cifrado TLS/SSL para conexiones externas"

_default: false

_rebootRequired: true

=====

2. GESTIÓN DE USUARIOS (COLECCIÓN DINÁMICA)

=====

users:

"{userId}":

_rule:

range: [0, 19] # Permite registrar hasta 20 usuarios externos

padding: 2 # Genera índices: 00, 01, 02...

fixed: false # Colección dinámica (se crean con POST, se borran con DELETE)

username:

```

    _type: string
    _description: "Nombre del usuario (ej. vms_integration)"
    _default: ""
    _maxLength: 32
    _forceSave: true

password:
    _type: password
    _description: "Contraseña segura para el broker"
    _maxLength: 64
    _policyLevel: 2    # Validación de complejidad delegada al Validator

# =====
# 3. PERMISOS ACL POR USUARIO (SUB-COLECCIÓN DINÁMICA)
# =====

acls:
    "{aclId}":
        _rule:
            range: [0, 9]    # Hasta 10 reglas de acceso por cada usuario
            padding: 2
            fixed: false

        permission:
            _type: list
            _description: "Nivel de acceso al Topic"
            _default: "read"
            _allowedValues:
                - "read"      # Solo suscribirse
                - "write"     # Solo publicar
                - "readwrite" # Suscribirse y publicar

        topic:
            _type: string
            _description: "Ruta del Topic (soporta wildcards + y #)"
            _default: ""
            _maxLength: 128

```

El motor rest renderiza su correspondiente vista de datos json:

```

{
  "server": {
    "enabled": "true",
    "port": "8883",
    "protocol": "mqtt",
    "tls_enabled": "true"
  },
  "users": {
    "00": {
      "username": "vms_integration",
      "password": "__reserved_pwd_",
      "acls": {
        "00": {

```

```

        "permission": "read",
        "topic": "nxt/alarms/#"
    },
    "01": {
        "permission": "write",
        "topic": "nxt/commands/#"
    }
}
},
"01": {
    "username": "cloud_sync",
    "password": "__reserved_pwd_",
    "acls": {
        "00": {
            "permission": "readwrite",
            "topic": "nxt/telemetry/#"
        }
    }
}
}
}
}

```

4.3 Registro mediante interfaz dbus

Esta sección documenta la **Interfaz D-Bus** del servicio de validación (`com.lanaccess.sysprop_validator`).

Esta interfaz actúa como el contrato de comunicación de bajo nivel entre los diferentes microservicios y el gestor central de configuración.

La siguiente lista define los métodos expuestos, sus firmas de entrada (argumentos) y salida (retorno).

"register_string_parameter",	"sa(siiib)sstbb", "b"
"register_password_parameter",	"sa(siiib)sstibb", "b",
"register_numeric_parameter",	"sa(siiib)ssbxbxbb"
"register_boolean_parameter",	"sa(siiib)sbbb", "b"
"register_list_parameter",	"sa(siiib)ssasbb", "b"
"register_flag_parameter",	"sa(siiib)ssasbb", "b"
"register_ipv4_parameter",	"sa(siiib)ssbb"
"register_ipmaskv4_parameter",	"sa(siiib)ssbb"
"register_resource_list_parameter",	"sa(siiib)sssiibb", "b"
"register_resource_flag_parameter",	"sa(siiib)sssiibb"
"set_default_options",	"sa(ss)s", "b"
"validate_parameter",	"ss", "(bs)"
"validate_parameter_list",	"a(ss)", "a(ss)"
"get_schema",	"ss", "s"
"is_registered",	"s", "b"
"version",	"", "s"
"shutdown",	"", ""

Convenciones de Tipos D-Bus

- `s`: String (cadena de texto UTF-8).
- `b`: Boolean (entero de 32 bits, 0 o 1).
- `i` / `x`: Enteros (32 / 64 bits).
- `a`: Array.
- `(...)`: Struct (agrupación de valores).

4.3.1. Métodos de Registro (`register_*_parameter`)

Estos métodos son utilizados por los microservicios al arrancar para declarar las propiedades que gestionan.

- **Argumentos Comunes:**

- `s` (path): Ruta de la propiedad (ej. `config.device.name`).
- `a(siiib)` (rules): Array de reglas para plantillas. Cada regla contiene:
 - `s`: Placeholder (ej. `"{id}"`).
 - `i, i`: Min/Max index.
 - `i`: Padding.
 - `b`: Fixed collection (true/false).
- `s` (description): Descripción humana.
- `s` (default): Valor por defecto.
- `b` (reboot): Si requiere reinicio (true/false).

- **Argumentos Específicos por Tipo:**

- `register_string...: t` (`max_length`).
- `register_password...: t` (`max_length`), `i` (`policy_level`).
- `register_numeric...: bxbx` (`tiene_min`, `min`, `tiene_max`, `max`).
- `register_list/flag...: as` (array de strings con valores permitidos).
- `register_resource...: s` (`prefix`), `ii` (`padding`, `max_count`).

4.3.2. Métodos de Validación (`validate_*`)

Utilizados por la API REST y la CLI antes de aplicar cambios.

- **`validate_parameter`:**

- Entrada: Clave, Valor.
- Salida: Struct (`b: es_valido`, `s: mensaje_error`).

- **`validate_parameter_list`:**

- Entrada: Array de pares (`Clave`, `Valor`).
- Salida: Array de errores (`Clave`, `Mensaje`). Si está vacío, todo es válido.

4.3.4 Métodos de Introspección (`get_schema`, `is_registered`)

Utilizados para descubrir la estructura y estado del sistema.

- **get_schema:**
 - Entrada: Path raíz, Formato ("json", "yaml").
 - Salida: String con el esquema serializado.
- **is_registered:**
 - Entrada: Clave.
 - Salida: True si la propiedad existe en el esquema.

4.3.5. Métodos de Control

- **version:** Devuelve la versión del servicio.
- **shutdown:** Ordena al servicio que se detenga de forma limpia.

4.3.6 Método `defaults` específicos (`set_default_options`)

Este método permite inyectar opciones avanzadas de valores por defecto en una propiedad ya registrada. Se utiliza para definir valores por defecto que dependen de la personalización del cliente (branding) o que deben ser calculados dinámicamente en tiempo de ejecución.

- **Firma de Entrada:** `sa(ss)s`
 1. `s` (String): **Path**. La ruta completa de la propiedad (ej. `config.network.port.http`).
 2. `a(ss)` (Array de Structs): **Custom Defaults**. Una lista de pares clave-valor, donde la clave es el identificador de la personalización (ej. "bbva", "caixa") y el valor es el default específico para ese cliente.
 3. `s` (String): **Default Function**. El nombre de una función C++ registrada en el servidor que será invocada para calcular el valor por defecto (útil para índices dinámicos, ej. "generate_camera_name").
- **Firma de Salida:** `b`
 1. `b` (Boolean): `true` si la operación fue exitosa (la propiedad existía y se actualizó), `false` en caso contrario.

4.3.6 API C++

Los diferentes módulos del firmware registran sus parámetros al arrancar usando la librería *header-only* `sysprop_client.hpp`.

- **Wrappers de Alto Nivel:** Se utilizan funciones C++ modernas (`register_numeric`, `register_flags_template`) que deducen tipos y rangos automáticamente.
- **Vectores como Fuente de Verdad:** Se pasan vectores de `std::pair<enum, string>` al registrar. El sistema deduce:
 - Los valores permitidos (`allowedValues`).
 - El valor por defecto (`default`).
 - El número máximo de elementos (para recursos).
- **Metadatos:** Cada propiedad incluye descripción, longitud máxima, rangos y el flag crítico `reboot_required`.

5. API REST (Interfaz para Angular)

La API REST actúa como puente inteligente entre el cliente (Aplicación Angular) y el núcleo del sistema. Su diseño se centra en trasladar la complejidad del procesamiento de datos del frontend al backend, entregando objetos JSON limpios, completos y listos para usar.

5.1. Lectura Inteligente (GET) - "Hydrated Data"

- **Endpoint:** `GET /config.device.alarms`
- **Objetivo:** Entregar una vista completa y coherente de la configuración, ocultando al cliente la complejidad del almacenamiento interno (que solo guarda los valores diferentes a los de defecto para ahorrar espacio).

5.1.1 El Proceso de Fusión (Merge Logic)

Cuando se solicita un nodo, el servidor realiza un proceso de "hidratación" en tiempo real:

1. **Carga de Datos:** Recupera del almacenamiento persistente las propiedades "crudas" que existen bajo el nodo solicitado.
2. **Carga de Esquema:** Consulta al Validator Service para obtener la definición estructural (esquema) de ese nodo.
3. **Construcción y Fusión de Árboles:**
 - Se construye un árbol en memoria con los datos existentes.
 - Se recorre el esquema en paralelo. Para cada propiedad definida en el esquema que falta en los datos, el sistema inyecta automáticamente su valor por defecto (`_default`).
 - Para las colecciones fijas (ej. Entradas físicas), el sistema genera automáticamente todas las instancias (01 a 16) aunque no estén configuradas.
4. **Resultado:** El frontend recibe un objeto JSON completo donde cada campo está presente. Esto elimina la necesidad de comprobaciones `if (data.field === undefined)` en el código del cliente.

5.1.2 Tipos y Seguridad en Lectura

- **Tipos:** Por simplicidad y consistencia, todos los valores primitivos (números y booleanos) se serializan como `string` en el JSON. El frontend utiliza el esquema para interpretarlos correctamente.
- **Contraseñas:** Las propiedades de tipo `password` nunca devuelven su valor real. Se sustituyen por el centinela `__reserved_pwd__` para indicar que la contraseña está configurada pero es secreta.

5.2. Escritura (PUT) - Edición Atómica

- **Endpoint:** PUT /config.device.users.user.00
- **Body:** JSON con el objeto a modificar.
- **Proceso:**
 1. Aplana el JSON a pares clave-valor.
 2. Ignora campos con valor `reserved_pwd` (no se tocan).
 3. **Valida en bloque** todas las propiedades contra el esquema.
 4. Si hay error, devuelve `400 Bad Request` con un array detallado de errores (`validationErrors`).
 5. Si todo es OK, realiza la escritura.

5.3. Creación (POST) - Colecciones

El flujo de trabajo es el siguiente:

1. **Discovery (Descubrimiento):** El cliente pide un nodo nuevo. El servidor consulta el esquema, fusiona los valores por defecto del nodo y entrega una "plantilla" completa.
2. **State Management (Gestión de estado):** El cliente modifica solo lo necesario.
3. **Persistencia Eficiente:** Al hacer **POST**, el servidor compara contra los valores por defecto y "limpia" el objeto antes de guardarlo en el almacén clave-valor. Solo guardas el **delta**.
4. **Reconstrucción Dinámica:** En la lectura, se realiza un *merge* en tiempo real.

- **Endpoint:** GET
/device-service/properties/template?node=config.device.users.user.0
- **Endpoint:** POST /config.device.users.user
- **Uso:** Crear una nueva instancia en una lista (ej. nueva alarma).
- **Lógica:**
 1. El servidor busca automáticamente el primer ID libre basándose en el esquema (`indexFrom`, `indexTo`, `padding`).
 2. Valida y guarda los datos iniciales.
 3. Devuelve `201 Created` y el `Location` del nuevo recurso.

5.4. Borrado (DELETE)

- **Endpoint:** `DELETE /config.device.users.user.00`
- **Uso:** Borrar una instancia o resetear una configuración a valores de fábrica.

5.5. Templates (GET /template)

Discovery (Descubrimiento): El cliente pide un nodo nuevo. El servidor consulta el esquema, fusiona los valores por defecto del nodo y entrega una "plantilla" completa.

- **Endpoint:** `GET /device-service/properties/template?node=config.device.users.user.0`
- **Uso:** Devuelve un objeto JSON "virgen" con todos los valores por defecto del esquema para ese tipo de objeto. Ideal para inicializar formularios de creación en Angular.

5.6. VISTAS (GET ?fields, PATCH ?replace) - PUT VIRTUAL

5.6.1. Vistas (GET ?fields)

Para el manejo de colecciones grandes en el que el número de elementos es grande y el tamaño por elemento también lo es (por ejemplo la configuración de 150 cámaras) se trabaja con vistas en lugar de objetos completos. Obtenemos así una vista para dar de alta y baja las cámaras, o una vista de la configuración de códecs de todas las cámaras, o vistas de grabaciones.

Ejemplo:

```
GET
config.video.cam?fields=base.ipcamera.connection|base.id&maxcount
=8&offset=8
```

Modificamos la vista:

```
PATCH config.video.cam?replace=base.ipcamera.connection
```

1. **Validación:** Igual que en PUT. Todo lo que se envía debe ser válido.
2. **Lógica Base (Merge):** A diferencia del PUT (que borra todo el nodo antes de escribir), el PATCH por defecto **sobrescribe** solo las propiedades que vienen en el JSON. Las que no vienen, se mantienen.
3. **Lógica** Si el usuario envía `?replace=subnodo1|subnodo2`, significa que para esos sub-nodos específicos, queremos comportamiento de PUT (borrar y reescribir) en lugar de merge. Esto es útil si quieres actualizar una lista completa o un objeto complejo dentro de un nodo más grande, asegurándote de eliminar cualquier propiedad antigua que no esté en el nuevo JSON para ese sub-nodo.

La metodología permite al cliente solicitar sólo un subconjunto específico de propiedades, reduciendo drásticamente el tamaño de la respuesta JSON.

- **Parámetro:** `fields` (lista separada por pipes `|`).
- **Comportamiento:** El servidor filtra el árbol de datos antes de serializarlo, devolviendo solo las ramas solicitadas y manteniendo la estructura jerárquica original.
- **Sintaxis de Filtrado:**
 - **Propiedad directa:** `fields=connection` (Devuelve el objeto `connection` completo de cada elemento).

- **Ruta profunda:** `fields=connection.ip|image.resolution` (Devuelve solo esos campos anidados).
- **Colecciones:** Si se solicita sobre una lista (ej. `.../camera`), el filtro se aplica a **cada elemento** de la lista.

Ejemplo: Obtener solo la IP y el Puerto de todas las cámaras.

`GET .../camera?fields=connection.ip|connection.port`

```
{
  "config.video.camera": {
    "00": { "connection": { "ip": "1.1.1.1", "port": "80" } },
    "01": { "connection": { "ip": "1.1.1.2", "port": "80" } }
    // ... resto de cámaras ...
  }
}
```

5.6.2. Edición Parcial y "PUT Virtual" (`PATCH`)

El método `PATCH` se utiliza para modificar una configuración sin tener que enviar el objeto completo. Es ideal para ediciones rápidas o actualizaciones masivas.

- **Comportamiento Base (Merge):** Por defecto, `PATCH` fusiona el JSON recibido con la configuración existente.
 - Las propiedades presentes en el JSON se actualizan.
 - Las propiedades ausentes en el JSON se **mantienen intactas**.
 - Para borrar (resetear a default) una propiedad específica, se debe enviar con valor igual al default.
- **Comportamiento de Reemplazo (** Permite simular un `PUT` (reemplazo total) pero limitado a sub-nodos específicos, dentro de una operación `PATCH` más grande.
 - **Parámetro:** `replace` (lista de sub-nodos separados por `|`).
 - **Lógica:** Para cada nodo listado en `replace`, el servidor **borra** todo su contenido antes de aplicar los nuevos valores del JSON.
 - **Uso:** Útil para actualizar listas complejas o estructuras donde es difícil calcular el "delta" en el cliente. "Quiero que la lista de servidores sea *exactamente* esta, borra los que sobren".

Ejemplo: Actualizar la configuración de red de la cámara 00, asegurando que cualquier configuración antigua de DNS se elimine si no se envía nueva.

```
PATCH .../camera.00?replace=connection.dns

{
  "connection": {
    "ip": "1.2.3.4",          // Se actualiza (Merge normal)
    "dns": {                  // Se REEMPLAZA totalmente (se borra lo anterior)
      "primary": "8.8.8.8"
    }
  }
}
```

Resultado: La IP cambia. La configuración DNS anterior se borra y se establece solo el primario. El resto de la cámara (imagen, ptz) no se toca.

5.6.3. Borrado Selectivo con Comodines (Wildcard Pruning) en PATCH

Cuando se utiliza el parámetro `replace` en una petición `PATCH`, el sistema debe determinar exactamente qué claves de configuración deben ser eliminadas antes de aplicar los nuevos valores.

Si la ruta especificada en `replace` contiene un comodín de plantilla (placeholder `{}`), el sistema ejecuta un algoritmo de **expansión de instancias** para garantizar un borrado completo y preciso.

Algoritmo de Poda:

1. **Carga del Estado Actual:** El sistema carga en memoria el árbol de configuración actual correspondiente al nodo base de la petición (`strPath`).
2. **Análisis de Rutas de Reemplazo:** Se itera sobre cada ruta especificada en el parámetro `replace` (ej. `media.video.{}.cond`).

3. **Navegación y Expansión:** El algoritmo recorre el árbol de datos actual siguiendo la ruta de reemplazo.
 - **Segmento Fijo (ej. "media"):** Se desciende al nodo hijo correspondiente.
 - **Segmento Comodín (** El algoritmo detecta que este segmento representa una colección. Itera sobre **todos** los hijos existentes en ese nivel del árbol de datos (ej. todas las cámaras 00, 01, 02... o todos los streams 0, 1...).
4. **Generación de Órdenes de Borrado:** Para cada rama terminal alcanzada tras la expansión, se genera una orden de borrado explícita.

Ejemplo Práctico:

- **Petición:** `PATCH .../config.video.cam?replace=media.video.{}.cond`
- **Estado Actual:** Existen 2 cámaras (00, 01), cada una con 2 streams (0, 1).
- **Expansión:** El comodín {} se expande primero para las cámaras y luego (si hubiera otro {}) para los streams.
- **Órdenes Generadas:** El sistema ejecutará internamente los siguientes borrados:
 1. `config.video.cam.00.media.video.0.cond.` (Borra condiciones del stream 0 de cam 00)
 2. `config.video.cam.00.media.video.1.cond.` (Borra condiciones del stream 1 de cam 00)
 3. `config.video.cam.01.media.video.0.cond.` (Borra condiciones del stream 0 de cam 01)
 4. `config.video.cam.01.media.video.1.cond.` (Borra condiciones del stream 1 de cam 01)

Esto garantiza que **todo lo viejo se elimina** en todas las instancias afectadas, dejando el sistema limpio para recibir la nueva configuración especificada en el cuerpo JSON, evitando así que queden "residuos" de configuraciones anteriores en instancias no explícitas.

5.7 Resumen de Estrategias de Escritura

Método	Objetivo	Comportamiento con datos no enviados	Uso Típico
PUT	Reemplazo Total	Se borran (resetean a default).	Edición de formularios completos.
PATCH	Edición Parcial	Se conservan .	Cambios rápidos (switches, sliders).
PATCH + <code>replace</code>	Reemplazo Parcial	Se borran solo en los nodos indicados.	Edición de sub-secciones complejas.

5.8. Vistas en detalle (`fields`)

Para nodos grandes (ej. lista de 150 cámaras), el cliente puede solicitar una proyección de los datos para ahorrar ancho de banda.

- **Uso:** `GET`
`config.video.cameras?fields=base.ipcamera.connection|base.ipcamera.account`
- **Lógica:** El servidor construye el árbol completo pero, antes de serializar a JSON, poda todas las ramas que no coincidan con la lista de campos solicitados, devolviendo un objeto ligero.

Esto permite al cliente solicitar sólo un subconjunto específico de propiedades o ramas del árbol de configuración, reduciendo el tamaño de la respuesta y el tiempo de procesamiento.

- **Parámetro:** `fields` (lista separada por pipes `|`).
- **Comportamiento:** El servidor procesa el árbol de datos resultante y elimina cualquier nodo que no coincida con los campos solicitados. La estructura jerárquica intermedia necesaria para llegar al dato se preserva automáticamente.

Sintaxis de Selectores:

El sistema soporta dos tipos de selectores de ruta:

1. **Ruta Exacta (Específica):** Selecciona una instancia concreta de un array o colección.
 - Ejemplo: `media.video.0.receiver`
 - *Efecto:* Devuelve solo la configuración del stream de vídeo 0.
2. **Ruta con Comodín (Genérica):** Utiliza `{}` para coincidir con **cualquier** índice en ese nivel.
 - Ejemplo: `media.video.{ }.receiver`
 - *Efecto:* Devuelve la configuración `receiver` para **todos** los streams de vídeo disponibles (0, 1, 2...).

Ejemplos de Uso Combinado:

- **Petición:** GET
`.../config.video.cam?fields=base.capabilities|base.read-only.encoder|media.video.{ }.receiver`
- **Resultado:** Para cada cámara, devuelve sus capabilities, el parámetro que indica si se permite configurar sus codecs y la configuración de cada receiver.

5.8.1 Limitación de Resultados (`maxcount`)

Para gestionar eficientemente colecciones grandes (ej. listas de cámaras), la API permite limitar el número de elementos devueltos en una única petición. Esto es esencial para optimizar el rendimiento del frontend y adaptarse a restricciones de licencia dinámicas.

- **Parámetro:** `maxcount` (entero positivo).
- **Comportamiento:** Si el nodo solicitado es una colección (un array de objetos indexados numéricamente), el servidor truncará la lista de resultados, devolviendo solo los primeros *N* elementos especificados.
- **Orden:** Los elementos se devuelven siempre ordenados por su índice (alfanumérico: "000", "001", "002"...), por lo que `maxcount=50` garantiza que se obtendrán las cámaras de la 1 a la 50.

Ejemplo de Uso:

Obtener la configuración básica de las primeras 16 cámaras para mostrar en una vista de cuadrícula.

```
GET .../config.video.cam?fields=base.ipcamera.connection.address
&maxcount=16
```


5.8.2 Paginado (offset+maxcount)

Para colecciones muy grandes se puede trabajar por páginas. Esto permitirá al frontend cargar listas largas "bajo demanda" o en páginas.

`/config.video.cam?fields=base.ipcamera.connection&maxcount=3&offset=2`

5.8.3 Filtrado Condicional (where)

Permite filtrar los elementos de una colección basándose en el valor de una de sus propiedades. Esto es fundamental para recuperar subconjuntos específicos de recursos (ej. "solo las cámaras activas", "solo los usuarios administradores").

1. **Parámetro:** `where`
2. **Sintaxis:** Estilo funcional ("Function Style").
 1. **Igualdad:** `eq(clave|valor)`
 - `clave`: Ruta relativa a cada elemento de la colección (ej. `base.installed`).
 - `valor`: Valor exacto esperado (ej. `true`, `RTSP/H264`).
 - *Nota*: Se usa `|` como separador para evitar conflictos con comas en valores complejos.
 2. **Pertenencia a Conjunto (Token):** `flagtoken(ruta|token)`
 - Diseñado para propiedades de tipo `flag` (listas separadas por `+`). Comprueba si el `token` especificado está presente en el valor de la propiedad.
 - *Ejemplo*: `flagtoken(base.capabilities|ptz)` coincidirá si el valor es `"osd+ptz+audio"`.
 3. **Límite de Índice:** `id_min_than(ruta|valor)`
 - Comprueba si el **ID numérico** del nodo en la ruta especificada es estrictamente menor que el valor dado.
 - Utilizar `{ }` como ruta para filtrar por el ID del elemento principal de la colección.
 - *Ejemplo 1*: `id_min_than({ }|5)` -> Filtra los elementos con ID 0, 1, 2, 3, 4.
 - *Ejemplo 2*: `id_min_than(media.video.{ }|2)` -> Filtra los elementos que tengan algún sub-elemento `video` con ID menor que 2.

4. **Operador Y:** `and(expresion1|expresion2|...)`
 - Todas las sub-expresiones deben ser verdaderas.
 5. **Operador O:** `or(expresion1|expresion2|...)`
 - Al menos una sub-expresión debe ser verdadera.
3. **Anidamiento:** Las expresiones se pueden anidar arbitrariamente para crear lógica compleja.
4. **Rutas con Comodines (Wildcards):**
1. Se puede utilizar `{ }` en cualquier parte de la ruta para indicar "cualquier elemento de esta colección".
 2. La lógica aplicada es **ANY (Existe alguno)**: La condición se considera verdadera si al menos un elemento de la sub-colección cumple el criterio.
 3. *Ejemplo:* `eq(media.video.{ }.multicast.use|true)`
 - Se traduce como: "¿Tiene esta cámara **algún** stream de vídeo (0, 1, 2...) que tenga multicast activado?".
5. **Interacción:** El filtro `where` se aplica **antes** que la paginación (`offset, maxcount`). Esto permite paginar correctamente sobre el conjunto filtrado.

Ejemplo de Consulta Compleja:

Recuperar las cámaras que están instaladas y que son de tipo "RTSP/H264" o "RTSP/H265". Además, proyectar solo sus capacidades y configuración de streams.

GET

```
.../config.video.cam?fields=base.capabilities|media.video.{ }.stream&offset=0&maxcount=10&where=and(eq(base.installed|true)|or(eq(base.ipcamera.type|RTSP/H264)|eq(base.ipcamera.type|RTSP/H265)))
```

```

"001": {
  "base": {
    "capabilities": "osd+motion+telemetry+secondEncoding+input+tamper+analytics"
  },
  "media": {
    "video": {
      "0": {
        "stream": { "port": "554", "uri": "stream1" }
      },
      "1": {
        "stream": { "port": "554", "uri": "stream2" }
      }
      // ...
    }
  }
}
// (Otras cámaras que no cumplan la condición son filtradas)
}

```

Otros ejemplos:

Recupera las cámaras ip que tengan configurado recibir algún flujo en multicast

GET

.../config.video.cam?fields=base.capabilities|media.video.{}.multicast&where=and(eq(media.video.{}.multicast.use|true)|eq(media.video.{}.receiver.transport|udp))

Recuperar las primeras 5 cámaras (ID < 5) que tengan capacidades PTZ.

GET

.../config.video.cam?where=and(id_min_than({}|5)|flagtoken(base.capabilities|ptz))

Esta capacidad permite al frontend realizar filtrados de "N primeros elementos" basándose en el ID lógico en lugar de la posición en la lista (que es lo que hace `maxcount`), ofreciendo un control más preciso.

[http://172.16.135.253/config.video.cam?fields=base.capabilities|media.video.{}.receive|recorder.{}.cond.type&where=and\(id_min_than\({}|150\)|or\(eq\(base.installed|true\)|eq\(base.ipcamera.src.child.allocated|false\)\)\)&maxcount=150&offset=0](http://172.16.135.253/config.video.cam?fields=base.capabilities|media.video.{}.receive|recorder.{}.cond.type&where=and(id_min_than({}|150)|or(eq(base.installed|true)|eq(base.ipcamera.src.child.allocated|false)))&maxcount=150&offset=0)

5.9. Sanitización y Autocuración de Datos (Pruning)

Para garantizar la robustez del sistema y la coherencia entre el esquema y los datos almacenados, la API REST implementa un mecanismo de **poda proactiva (pruning)** en las operaciones de lectura (`GET`).

- **Problema:** Con el tiempo, o debido a intervenciones manuales, la base de datos de propiedades puede contener claves obsoletas (renombradas o eliminadas en nuevas versiones del firmware) o inválidas ("basura"). Si estos datos se entregaran al cliente, un posterior `PUT` fallaría la validación estricta, bloqueando al usuario.
- **Solución:** Antes de entregar cualquier respuesta `GET`, el sistema compara el árbol de datos recuperado con el árbol de esquema actual.
 1. Cualquier propiedad o rama presente en los datos que **no exista en el esquema** es eliminada automáticamente de la respuesta en memoria.
- **Beneficio:**
 1. **Limpieza Transparente:** El frontend nunca recibe datos obsoletos.
 2. **Autocuración:** Al realizar el siguiente `PUT` con los datos limpios recibidos, las propiedades basura se eliminan permanentemente de la base de datos.

Ejemplo:

- Datos en propiedades: `{"ip": "1.1.1.1", "old_param": "basura"}`
- Esquema Actual: Solo define `ip`.
- Respuesta `GET`: `{"ip": "1.1.1.1"}` (`old_param` ha sido podado).

5.10. Gestión de Valores Vacíos y el Centinela de Estado (Empty State Sentinel)

Al mapear un modelo de datos jerárquico y rico (JSON) sobre un sistema de almacenamiento persistente plano de tipo clave-valor (D-Bus / Core Legacy), surge una fricción arquitectónica fundamental respecto al manejo de los "valores vacíos".

La Problemática: "Borrar" vs "Vaciar"

En los sistemas clave-valor tradicionales, enviar una petición de escritura con un valor vacío (ej. `config.alarms.01.action=""`) es el comando universal que interpreta la base de datos para **eliminar esa clave del almacenamiento**.

Sin embargo, desde el punto de vista de la lógica de negocio (el Esquema), un string vacío "" a menudo representa un **estado válido y explícito** que difiere del valor por defecto del sistema.

El Caso de Uso Conflictivo:

Imaginemos una propiedad de tipo `flag` (selección múltiple) que define las acciones de una alarma.

1. El esquema define que el valor por defecto es `"record+relay"`.
2. El usuario, a través de la interfaz web, decide deshabilitar todas las acciones para esa alarma y envía un `PUT` o `PATCH` con `"action": ""`.
3. El backend procesa la petición, ve que "" es diferente al default, y decide que debe guardarlo. Envía el comando `action=""` a la base de datos D-Bus.
4. La base de datos, siguiendo su protocolo interno, **borra** la clave `action` del disco.
5. Cuando el frontend realiza un `GET` para refrescar la vista, el motor de hidratación (`merge_trees`) lee la base de datos, ve que la clave `action` no existe, y deduce: *"Como no existe, el usuario no la ha configurado. Le inyecto el valor por defecto"*.
6. El frontend recibe `"action": "record+relay"`. **El sistema ha ignorado la voluntad del usuario de vaciar la propiedad.**

La Solución: Desacoplamiento mediante el Centinela `__is_empty__`

Para resolver este conflicto sin alterar el comportamiento de bajo nivel de la base de datos D-Bus, el sistema NXT introduce el concepto de **Centinela de Estado Vacío** `__is_empty__`.

Esta solución desacopla el concepto de "borrado de base de datos" del concepto de "valor de negocio vacío", interceptando los datos en las capas de entrada y salida de la API REST.

Flujo de Trabajo del Centinela:

1. Escritura (PUT / PATCH / POST):

Cuando el cliente envía explícitamente una propiedad con valor "", el sistema consulta el esquema. Si ese valor vacío difiere del valor por defecto y, por tanto, debe persistir, la API REST inyecta automáticamente el centinela y envía a D-Bus el valor de la propiedad: `__is_empty_`.

La base de datos lo interpreta como un string válido y lo guarda en disco, preservando la intención del usuario.

2. Lectura (GET):

Durante la extracción de datos desde D-Bus, el sistema escanea todos los valores. Si encuentra el texto `__is_empty_`, lo traduce inmediatamente en memoria a un string vacío real "".

A continuación, cuando actúa el motor de hidratación, detecta que la propiedad *sí existe* y tiene el valor "", por lo que **se abstiene de inyectar el valor por defecto**. El frontend recibe un JSON impecable con "action": "".

3. Restauración a Valores de Fábrica:

Si el usuario realmente desea restaurar la propiedad a su estado de fábrica (ej. que vuelva a ser "record+relay"), el frontend simplemente debe enviar en el PATCH el valor "record+relay". El backend detectará que coincide exactamente con el `_default` del esquema y enviará un "" (vacío real) a D-Bus. Esto provocará el borrado físico de la clave, limpiando la memoria Flash y permitiendo que el valor por defecto vuelva a emerger de forma natural en futuras lecturas.

4. Prevención de Falsos Reinicios:

El motor inteligente de detección de reinicios contempla esta centinela. Al comparar el JSON entrante ("") con el estado actual de la base de datos (`__is_empty_`), el motor sabe que semánticamente son idénticos y evita marcar el equipo para un reinicio innecesario.

Conclusión:

Mediante el uso de `__is_empty_` y el enmascaramiento de contraseñas (`__reserved_pwd_`), la API REST actúa como un escudo protector semántico. Garantiza que la base de datos plana se mantenga optimizada y limpia (guardando sólo las deltas), al mismo tiempo que respeta escrupulosamente las intenciones de configuración complejas del usuario y del esquema.

6. API de Introspección (Schema API)

El sistema expone una API para que los clientes puedan descubrir dinámicamente la estructura, tipos, validaciones y documentación de la configuración. Esto es fundamental para que el Frontend (Angular) pueda generar formularios automáticamente sin tener conocimiento "hardcodeado" del modelo de datos. En nuestro caso hemos optado por un enfoque híbrido: se renderiza automáticamente los parámetros pero elegimos cuales ponemos en cada pantalla.

Endpoint: `GET /device-service/properties/schema`

- **Parámetro:** `node` (Path del nodo a consultar).
- **Respuesta:** Un objeto JSON que describe la estructura del nodo solicitado y sus hijos.

Estructura del JSON de Esquema

El JSON devuelto sigue una estructura jerárquica idéntica a la de los datos, pero cada nodo hoja (propiedad) es reemplazado por un objeto de metadatos que describe esa propiedad.

Metadatos Comunes (Prefijo `_`)

Todas las claves de metadatos comienzan con `_` para evitar colisiones con nombres de propiedades reales.

- `_type`: El tipo de dato (`string`, `numeric`, `boolean`, `list`, `flag`, `resource_flag`, etc.).
- `_description`: Texto explicativo para mostrar en la interfaz (tooltip o label).
- `_default`: El valor por defecto del sistema.
- `_rebootRequired`: `true` si cambiar este valor requiere reiniciar el equipo.
- **Restricciones:** `_minValue`, `_maxValue`, `_maxLength`, `_allowedValues`.

Plantillas y Colecciones (`{placeholder}`)

Cuando un nodo representa una colección dinámica (como una lista de eventos o alarmas), se utiliza una clave especial entre llaves (ej. `"{eventId}"`).

- `_rule`: Objeto que define las reglas de generación de la colección.
 - `indexFrom`, `indexTo`: El rango numérico válido para los índices.
 - `padding`: El formato del índice (ej. 2 significa que los índices son "00", "01"...).
 - `fixed`: `true` si la colección es estática (hardware), `false` si es dinámica (creable/borrable).

Ejemplo Explicado

Petición:

GET .../schema?node=config.io.outputs.{}.task.cond.ev1

```
{
  "config.io.outputs.{}.task.cond.ev1": {
    // Definición de la plantilla para los eventos de la condición
    "{eventId}": {

      // Regla de índice: Hay 10 eventos posibles (0-9), con formato "00", "01"...
      "_rule": {
        "indexFrom": 0,
        "indexTo": 9,
        "padding": 2,
        "fixed": false
      },

      // Propiedad: Cámaras con analítica (Tipo Resource Flag)
      "analytic-cameras": {
        "_type": "resource_flag",
        "_description": "On external intelligent device trigger",

        // Formato del recurso subyacente (ej. analytic01, analytic02...)
        "_format": "analytic%00d",

        // Nota informativa sobre los IDs de recurso válidos
        "_rangeNote": "[1, 152] where N is the max number of resources..."
      },

      // Propiedad: Estado del DVR (Tipo Flag simple)
      "dvr-status": {
        "_type": "flag",
        "_description": "DVR Status",
        "_allowedValues": [ // Lista cerrada de opciones combinables
          "dvr-armed",
          "dvr-idle",
          "dvr-alarm-on",
          "dvr-alarm-idle"
        ]
      }
    }

    // ... resto de propiedades del evento ...
  }
}
```


7. API de Recursos (/resources)

Expone información sobre las capacidades del sistema que no son configuración, sino hechos.

El enfoque de **Resources** como un REST complementario resuelve el problema de la "Configuración Fantasma". Evita que el usuario pueda programar una grabación por eventos de un motion que no ha sido configurado.

- `GET /device-service/resources/video/cameras`: Devuelve array `[1, 2, 5]` indicando qué cámaras están instaladas físicamente.
- `GET /device-service/resources/clock/timezones`: Devuelve el árbol de zonas horarias.
- `GET /device-service/resources/clock/datetime`: Devuelve la hora local, zulu y si el equipo está configurado en ntp o manual.
- `PUT /device-service/resources/clock/datetime`: Orden para configurar la hora. Acepta local o zulu.
- `GET /device-service/resources/users/map/profiles`: Devuelve qué permisos (`grants`) tiene cada perfil de usuario.
- `GET /device-service/resources/io-space`: devuelve las entradas y salidas instaladas.
- `GET /device-service/resources/extmod`: devuelve las entradas y salidas detectadas en el módulo externo.
- `GET /device-service/resources/condition/events`: devuelve todas las entradas para que se pueda renderizar los eventos de una condición según los recursos instalados..
- `GET /device-service/resources/capabilities`: devuelve las capacidades del sistema. Por ejemplo si tiene test de baterías, número de cámaras licenciadas, si hay alguna cámara rtsp, si hay alguna cámara en multicast..
- `GET /device-service/resources/video/ipcameras/info`: devuelve el estado de conexión con las cámaras ip, fabricante, modelo, versión...
- `GET /device-service/resources/video/ipcameras/video_encoder_configuration_options?available=true&detailed=true`: devuelve cuantos codecs tiene realmente la cámaras y sus [capacidades](#). Si se indica `available=true` se muestran solo las cámaras actualmente detectadas. Si se indica `detailed=true` se muestran todas las opciones de cada codec(resoluciones, frame rates permitidos, intervalor de gop, etc).
- `GET /device-service/resources/video/ipcameras/audio_encoder_configura`

`tion_options?available=true/false`: devuelve las cámara que soportan audio y la info del codec de audio.

- `GET`
`/device-service/resources/video/ipcameras/metadata_configuration_options?available=true/false`: devuelve las cámaras que soportan metadatos de onvif.
- `GET`
`/device-service/resources/video/ipcameras/capabilities?available=true/false`: devuelve las capabilities de las cámaras, si la cámara soporta video, audio, metadata, entrada física, salida digital, tamper, motion, analytics, audio_detection, daynight_detection.
- `GET /device-service/resources/video/ipcameras/videosources`: devuelve las cámaras multicanal con sus canales.
- `GET`
`/device-service/resources/video/ipcameras/events_report?idr=000`: devuelve las subscripciones a las cámaras con los eventos que publica indicando si los soportamos o no.
- `GET /device-service/resources/video/ipcameras/lock`: devuelve si las fuentes de video están bloqueadas.
- `PUT /device-service/resources/video/ipcameras/lock`: bloquea las fuentes de video. Si se le pasa el argumento `force=true` bloqueará las fuentes aunque alguna cámara interna del switch se esté leyendo. Sin el argumento `force` devuelve error si alguna cámara interna del switch se está aún leyendo.
- `GET /device-service/resources/hd`: devuelve los discos configurados, detectados y el estado del raid
- `PUT /device-service/resources/hd/format`: formatea los discos que procedan en función del raid configurado.

Esto crea un sistema de validación en tres capas:

1. **Capa Teórica (Esquema):** "El sistema admite hasta 32 entradas remotas".
2. **Capa Física (Resources):** "Este equipo tiene un dispositivo con 6 entradas remotas".
3. **Capa Lógica (Configuración):** "Quiero grabar cuando se active la entrada remota 3".

Validación en Cascada

Esto te permite la siguiente lógica de validación:

- **Validación de Tipo:** ¿Es un número? (Capa Esquema).
- **Validación de Rango:** ¿Está entre 1 y 32? (Capa Esquema).
- **Validación de Existencia:** ¿Esa entrada está presente físicamente ahora mismo? (Capa Resources).

La aplicación angular es quien consulta los recursos para solo mostrar lo realmente existente. Si el usuario intenta hacer un **PUT** para configurar la grabación en función de la entrada 10, el backend puede responder con un error 422 (Unprocessable Entity) diciendo: *"Válido según esquema, pero inexistente según recursos"*. Ahora lo controla el cliente angular y no se dará el caso.

El Esquema te da las reglas del juego (tipos de datos, límites teóricos).

Los Recursos te dan el tablero real (qué canales están online, qué sensores existen).

El Nodo de Datos te da la jugada actual (qué ha configurado el usuario).

8. Seguridad y Concurrency

8.1. Sistema de Bloqueo (Locking)

El sistema implementa un mecanismo de bloqueo pesimista jerárquico para garantizar la integridad de los datos en un entorno multi-usuario y proteger contra ataques de falsificación de peticiones (CSRF).

Objetivos

1. **Concurrency:** Evitar condiciones de carrera donde dos usuarios (o dos pestañas) intentan modificar la misma configuración simultáneamente.
2. **Seguridad:** Proporcionar un token anti-CSRF vinculado a la sesión de edición.
3. **Experiencia de Usuario:** Informar inmediatamente al usuario si un recurso está siendo editado por otro, evitando la frustración de guardar y recibir un error.

Flujo de Trabajo

Solicitud de Bloqueo

- **Endpoint:** GET
`/device-service/properties/lock?node=...&seed=...`
- **Parámetro** El path del recurso a bloquear (ej. `config` o `config.device`).
- **Parámetro** Un identificador único generado por el frontend (Angular) y almacenado en `sessionStorage` (que es único por pestaña).
- **Respuesta Exitosa (200 OK):**

```
{
  "token": "aVQ3wjNZ5QIXNAGjhfGgYdpFDWoHsJW5",
  "expiresIn": 120,
  "path": "config"
}
```

Respuesta de Conflicto (409 Conflict): Si el recurso ya está bloqueado por otro usuario o por otra pestaña del mismo usuario (diferente seed).

```
{
  "cause": "ResourceLocked",
  "detail": "Resource is locked by user 'admin' (IP 10.0.0.1)",
  "lockedPath": "config"
}
```

Renovación Automática:

- Si un cliente solicita un bloqueo sobre un recurso que **él mismo posee** (mismo Usuario, misma IP y misma `seed`), el servidor renueva el tiempo de expiración y devuelve el mismo token (o uno nuevo válido). Esto permite refrescar la página (F5) sin perder el control.

Operaciones de Escritura

- **Requisito:** Todas las peticiones de modificación deben incluir el parámetro `token` en la query string.
- **Validación:** El servidor verifica que:
 1. El token es válido y existe.
 2. El token corresponde a un bloqueo activo sobre el nodo solicitado o uno de sus ancestros.
 3. La petición proviene de la misma IP y Usuario que creó el bloqueo original.
- Si la validación falla, se devuelve `403 Forbidden`.

Liberación de Bloqueo

- El cliente puede liberar el bloqueo explícitamente llamando a `GET /unlock?node=...&token=...`
- Los bloqueos también expiran automáticamente tras un tiempo de inactividad (`expiresIn`) para evitar recursos bloqueados indefinidamente.

Protección de Pestañas Múltiples

Gracias al parámetro `seed`, el sistema distingue entre diferentes pestañas del navegador del mismo usuario.

- **Pestaña A:** Solicita lock con `seed="A"`. Obtiene token. Puede editar.
- **Pestaña B:** Solicita lock con `seed="B"`. El servidor detecta que el recurso está bloqueado por `seed="A"`. Deniega el bloqueo. La pestaña entra en modo **Solo Lectura**.

8.2. Contraseñas

- **Write-Only:** Nunca se envían al cliente.
- **Cambio Seguro:** Endpoint específico `PUT .../superuser/password` que requiere la contraseña actual y la nueva para validar.

9. Gestión de Reinicio (Reboot Required)

El sistema detecta automáticamente cuando una configuración crítica ha sido modificada y alerta al usuario de que es necesario un reinicio para aplicar los cambios.

1. Definición en el Esquema:

Cada propiedad puede tener el atributo `_rebootRequired: true` en su definición.

```
ip:
  _type: ipaddr
  _rebootRequired: true
```

2. Detección en Escritura (PUT/PATCH):

Al procesar una modificación, el servidor consulta el esquema. Si alguna de las propiedades modificadas tiene `_rebootRequired: true`, el sistema activa internamente un flag global de "Reinicio Pendiente".

3. Notificación al Cliente

Todas las respuestas exitosas de la API (GET, PUT, POST, DELETE) incluyen una cabecera HTTP personalizada que indica el estado actual del sistema.

- a. `X-Reboot-Required: true` -> El sistema tiene cambios pendientes de aplicar. El frontend debe mostrar una alerta.
- b. `X-Reboot-Required: false` -> El sistema está actualizado.

Esta arquitectura desacopla la lógica: el backend sabe **qué** requiere reinicio, y el frontend sabe **cuándo** avisar al usuario, sin tener que duplicar la lógica de negocio.

10. Notificaciones

El sistema envía un evento Dbus cuando se realiza un cambio en la configuración ya sea desde el api REST y la consola para que el resto de microservicios puedan reaccionar al cambio.

Evento dbus:

"CONFIG_CHANGED", "s", s

- **Argumentos:**
 - s (path): Ruta de la propiedad o nodo modificado (ej.
`{config.device.name},`
`{network.ports},{network.if},{config.video.cam}`).

11. Validación de Integridad Cruzada (Semantic Cross-Validation)

El sistema de gestión de configuración se apoya en el *Validator Service* (basado en el esquema YAML) como primera línea de defensa para garantizar la corrección atómica de los datos (tipos, rangos, formatos de IP, longitudes máximas).

Sin embargo, en sistemas de alta complejidad, surgen escenarios donde la validez de un parámetro depende estrictamente del valor de otro parámetro dentro de la misma estructura. Para mantener el esquema YAML declarativo, rápido y libre de lógica de programación (Single Source of Truth), la **validación relacional o cruzada** recae en el motor de la API REST. Al estar en el mismo proceso, intercepta la petición antes de que toque la persistencia.

11.1. La Problemática: Dependencias Semánticas

Un ejemplo clásico es la configuración de red (Ethernet). El *Validator Service* puede garantizar que la dirección IP asignada al **Gateway** (`config.network.if.eth0.gateway`) tiene un formato válido (ej. `192.168.1.1`). Pero, ¿es este **Gateway** alcanzable desde las IPs configuradas en la interfaz?

Si un administrador configura la interfaz `eth0` con la IP `10.0.0.50` y máscara `255.255.255.0`, y simultáneamente intenta asignar el **Gateway** `192.168.1.1`, el sistema quedaría en un estado de red inconsistente (el **Gateway** está fuera de la subred).

El esquema YAML, al ser declarativo y evaluar nodo por nodo, no tiene la capacidad matemática ni el contexto global para realizar esta comprobación de enrutamiento.

11.2. El Concepto: Validación de "Estado Futuro"

Para resolver este problema sin violar la transaccionalidad, la API REST implementa un motor de **Simulación en Memoria**. A diferencia de la validación individual, la validación cruzada no inspecciona el JSON de entrada del usuario de forma aislada, sino que construye una maqueta exacta de cómo quedará el sistema si se acepta la operación.

La construcción de este *Future State* es un proceso algorítmico riguroso que garantiza que el validador tenga la "foto real":

1. **Carga Base:** Se carga en memoria el estado actual del dispositivo desde la base de datos (D-Bus).

2. **Simulación de Poda (Wildcard Pruning):** Si la petición es un `PATCH` con directivas de reemplazo (`?replace=...`), el motor simula el borrado de las ramas correspondientes en memoria.
3. **Fusión (Merge):** Se inyectan los datos nuevos solicitados por el usuario.
4. **Desencriptación Contextual:** Si el usuario envió campos con el centinela `__reserved_pwd__` (indicando que no desea cambiar la contraseña), el motor rescata la contraseña real de la base de datos y la inyecta en la simulación. Esto garantiza que si un validador necesita comprobar credenciales cruzadas (ej. "No se puede activar el servicio si la contraseña está vacía"), disponga del dato real.

Una vez construido este mapa consolidado, se invoca a los validadores. Si alguno detecta una inconsistencia lógica, la transacción se aborta inmediatamente, devolviendo un error `HTTP 400 Bad Request` detallado.

11.3. Disparo Bidireccional (Ancestros y Descendientes)

Un reto arquitectónico fundamental en las APIs jerárquicas es determinar **cuándo** debe ejecutarse un validador cruzado. Si un validador está programado para proteger la ruta `config.network.if.`, el motor de ejecución aplica una lógica de intersección espacial bidireccional para garantizar que ninguna vía de entrada evada la seguridad:

- **Micro-operaciones (Ataque Descendiente):** Si el usuario modifica una propiedad específica mediante `PATCH /config.network.if.eth0.ip`, el motor detecta que la ruta solicitada es un *descendiente* de la ruta protegida, y dispara el validador.
- **Macro-operaciones (Ataque Ancestro):** Si un administrador sube un archivo de backup completo al equipo mediante `PUT /config`, la ruta solicitada ("`config`") no coincide textualmente con el validador de red. Sin embargo, el motor detecta que la petición es un *ancestro* estructural de la ruta protegida. El validador se despierta automáticamente, extrae su porción de red del JSON gigante, y la valida.

Esta arquitectura garantiza que las reglas de negocio sean **inquebrantables**, ya sea editando un solo switch en la interfaz web o restaurando masivamente una flota de grabadores.

11.4. Arquitectura del Registro de Validadores

Siguiendo la misma filosofía modular que el sistema de recarga post-escritura (`reloaders`), los validadores cruzados se registran mediante *callbacks* asociados a una ruta base.

Por ejemplo, cuando llega una petición `PUT` o `PATCH` sobre `config.network.if`, el motor de la API detecta la coincidencia de prefijo y ejecuta `validate_ethernet_if`.

11.4. Ejemplo Práctico: Validación de Gateway y Subredes Multihoming

El grabador NXT soporta múltiples direcciones IP estáticas por interfaz (*multihoming*). El validador cruzado de red debe comprobar si el *Gateway* propuesto es compatible con *al menos una* de las subredes configuradas.

El *callback* extrae el *Gateway* del `future_state`. A continuación, itera sobre los 8 *slots* posibles de configuración IP (`ip-user.{id}.addr` y `ip-user.{id}.mask`). Utilizando aritmética binaria estándar $((IP \& Mask) == (Gateway \& Mask))$, verifica la alcanzabilidad. De igual manera se comprueba que si ambos interfaces `lan` no están en `dhcp` entonces se comprueba que no haya solape de direcciones ip.

11.5. Beneficios Arquitectónicos

- **Protección ante operaciones parciales (PATCH):** Si el usuario modifica exclusivamente la máscara de red mediante un comando `PATCH`, el validador cruzado comparará esta nueva máscara contra la IP y el *Gateway* que **ya existen** en la base de datos, abortando la operación si el cambio rompe la topología de red.
- **Centralización de la Lógica de Negocio:** Al igual que con el esquema YAML, el frontend (Angular) no necesita replicar reglas complejas de validación de subredes. Si el backend rechaza la operación, el frontend simplemente muestra el mensaje `error_msg` devuelto por la API.
- **Escalabilidad:** Añadir nuevas validaciones cruzadas (ej. *"No se puede activar el servidor FTP si el puerto 21 está siendo usado por la telemetría"*) requiere únicamente añadir una función C++ y registrarla en el vector de callbacks.

11.6. Lista de validaciones cruzadas implementadas:

- **Topología Multihoming (LAN1 / LAN2):**
 - Comprobación matemática de que el *Gateway* configurado es alcanzable a través de al menos una de las direcciones IP y máscaras asignadas a la interfaz.
 - Comprobación de solapamiento de redes: Garantiza que ninguna de las subredes configuradas en LAN1 colisione con las subredes de LAN2 (a menos que operen bajo DHCP, donde la validación se delega al servidor externo).
- **Unicidad de Puertos:** Verificación en bloque para garantizar que no se asignan puertos de escucha duplicados entre diferentes servicios de entrada.

- **Integridad de Usuarios:** Comprobación de que al crear o editar el bloque de cuentas, no existan nombres de usuario repetidos en la colección activa.

12. Consola de Administración (CLI)

Interfaz interactiva accesible por SSH/Telnet.

- **Navegación:** Comandos `ls` (con colores para distinguir nodos/hojas) y `cd` para moverse por el árbol de configuración.
- **Edición:** Comando `set variable valor`. Validación en tiempo real.
- **Ayuda:** Comando `schema` muestra la definición YAML del nodo actual (tipo, rango, descripción) para ayudar al operador.
- **Listas:** Comando `add` detecta automáticamente el siguiente ID libre y entra en el contexto.

La consola SSH pueda "navegar" por las propiedades significa que tenemos un motor de introspección real. La consola no tiene los comandos programados a fuego; "lee" el esquema y construye el árbol de navegación al vuelo.

Consistencia Total: No hay riesgo de que un parámetro sea configurable por SSH pero no por la web, o que los rangos permitidos sean distintos. El **Esquema** manda sobre todos los canales.

Mantenibilidad: Si un desarrollador de un microservicio añade la propiedad `config.vpn.protocol`, no tiene que hablar con el equipo de la Consola, ni con el del Backend REST, ni con el del Frontend. Solo registra la propiedad y **aparece en todos lados**.

```
HARDWARE: LANACCESS NX1
FIRMWARE: 1.0.0_x241
MAC:      7ecfee:0c3640
FECHA:    26/02/2026 16:30:08
MONTAJE NO DETECTADO !!
SYSTEM LICENSE: OK

Username: hello
Password: *****

sys >cfg
Ok
cfg >enter
(config) >ls
device/
io/
network/
services/
video/
Total: 5 items

(config) >
```

```
(config) >cd video

(config.video) >ls
analog/
cam/
camera/
device/
map/
onvif/
recordings/
transmitters/
video-engine/
Total: 9 items

(config.video) >
```

```
(config.video.cam) >ls
000/
001/
002/
003/
004/
005/
006/
007/
008/
```

```
(config.video.cam) >cd 000
(config.video.cam.000) >ls
base/
media/
recorder/
telemetry/
transmitter/
xfeature/
Total: 6 items
(config.video.cam.000) >
```

```
(config.video.cam.000) >cd media
(config.video.cam.000.media) >ls
audio/
metadata/
video/
Total: 3 items
(config.video.cam.000.media) >
```

```
(config.video.cam.000.media) >cd video
(config.video.cam.000.media.video) >ls
0/
1/
2/
Total: 3 items
(config.video.cam.000.media.video) >
```

```
(config.video.cam.000.media.video) >cd 0
(config.video.cam.000.media.video.0) >ls
cond/
cropping/
enable-eptz = false
flush-min-time = 10
force-codification-to = NOFORCE
modes/
multicast/
receive = true
receiver/
receiver-path =
receiver-port = 554
receiver-state = init
stream/
Total: 13 items
(config.video.cam.000.media.video.0) >
```

```
(config.video.cam.000.media.video.0.modes) >ls
0/
1/
Total: 2 items
(config.video.cam.000.media.video.0.modes) >
```

```
(config.video.cam.000.media.video.0.modes.0) >ls
bandwidth = 2Mbps
frame-rate = 25
gop-size = 24
height = 0
priority = quality
quality = 22
resolution = 1920x1080
type = idle
width = 0
Total: 9 items
(config.video.cam.000.media.video.0.modes.0) >
```

```
(config.video.cam.000.media.video.0.modes.0) >schema bandwidth
---
config.video.cam.{}.media.video.{}.modes.{}.bandwidth:
  _type: list
  _description: "Bandwidth"
  _default: 2Mbps
  _allowedValues:
    - 20kbps
    - 30kbps
    - 40kbps
    - 50kbps
    - 64kbps
    - 128kbps
    - 256kbps
    - 350kbps
    - 512kbps
    - 768kbps
    - 1Mbps
    - 1.25Mbps
    - 1.5Mbps
    - 1.75Mbps
    - 2Mbps
    - 2.25Mbps
    - 2.5Mbps
    - 2.75Mbps
    - 3Mbps
    - 4Mbps
    - 6Mbps
    - 8Mbps
```

```
(config.video.cam.000.media.video.0.modes.0) >schema gop-size
---
config.video.cam.{}.media.video.{}.modes.{}.gop-size:
  _type: numeric
  _description: "GOP Size"
  _default: 24

(config.video.cam.000.media.video.0.modes.0) >
```

```
(config.video.cam.000.media.video.0.modes.0) >help
Available Commands:
ls
    List properties and child nodes in the current context.
cd <node|../>
    Change current context. Use '..' for parent, '/' for root.
set <key> <value>
    Set a property value. Use quotes for values with spaces.
    Example: set name "My Device Name"
default <key>
    Reset a property to its default value.
add [id]
    Add a new item to a list (e.g., a new camera or user).
    If [id] is omitted, the next available ID is used.
rm <node>
    Delete an entire node or item (recursive).
    Example: rm 00 (deletes alarm 00)
schema [node]
    Show the YAML schema definition for the current context or a child node.
exit
    Exit configuration mode.

(config.video.cam.000.media.video.0.modes.0) >
```

```
(config.video.cam.000.media.video.0.modes.0) >set bandwidth 8000
Validation Error: The key 'config.video.cam.000.media.video.0.modes.0.bandwidth' does not exist in the scheme.

(config.video.cam.000.media.video.0.modes.0) >
```

```
(config.video.cam.000.media.video.0.modes.0) >set bandwidth 8000
Validation Error: The value '8000' is not a valid option. Allowed: [20kbps, 30kbps, 40kbps, 50kbps, 64kbps, 128kbps, 256kbps,
5, 2.5Mbps, 2.75Mbps, 3Mbps, 4Mbps, 6Mbps, 8Mbps].

(config.video.cam.000.media.video.0.modes.0) >
```

```
(config.video.cam.000.media.video.0.modes.0) >set bandwidth 1Mbps
Property set successfully.

(config.video.cam.000.media.video.0.modes.0) >
```


13. Comando sprops

Igualmente disponemos del comando sprops para gestionar las propiedades dando el path absoluto del nodo o la propiedad:

```
sys >sprops --help
```

NAME

sprops - manage and display system properties

SYNOPSIS

```
sprops [--help]
sprops
sprops --tree <LEVEL>
sprops [-s] --ctx <CONTEXT>
sprops --reset <CONTEXT>
sprops --delete <PROPERTY>
sprops --set <PROPERTY> <VALUE>
sprops --schema <CONTEXT>
sprops --validate <PROPERTY> <VALUE>
```

DESCRIPTION

The sprops command is used to display, manage, and delete system properties. Properties are organized in a hierarchical, dot-separated key-value structure (e.g., config.device.name = mydevice).

When invoked without arguments, sprops lists all system properties and their current values.

OPTIONS

--help

Display this help message and exit.

--tree <LEVEL>

Display the properties as a tree, up to the specified LEVEL of depth. A LEVEL of 0 shows only the root nodes.

--ctx <CONTEXT>

Display all properties and their values under the specified CONTEXT (a property prefix, e.g., 'config.device').

-s, --structured

Used in conjunction with --ctx. Displays the properties within the context in a structured tree format instead of a flat list.

--reset <CONTEXT>

Deletes all properties under the specified CONTEXT. This action is recursive and cannot be undone.

`--delete <PROPERTY>`
Deletes a single, fully-qualified PROPERTY
(e.g., 'config.device.name').

`--set <PROPERTY> <VALUE>`
set a single, fully-qualified PROPERTY
(e.g., 'config.device.name').

EXAMPLES

List all properties
`sys> sprops`

Show a structured tree of all device configuration
`sys> sprops -s --ctx config.device`

Delete the specific property for the last boot mode
`sys> sprops --delete sys.boot.last_mode`

Reset all networking configuration
`sys> sprops --reset config.network`

Set device name
`sys> sprops --set config.device.name "mi equipo"`

14. Ejemplo de Flujo de Datos (JSON)

Esquema (Definición):

JSON

```
"scope": {  
  "{id}": {  
    "_rule": { "indexFrom": 0, "indexTo": 4, "padding": 2 },  
    "key": { "_type": "string", "_default": "" }  
  }  
}
```

Datos Almacenados (Interno):

```
config.device.onvif.scope.00.key = "MiClave"
```

Salida API REST (Frontend):

None

```
"scope": {  
  "00": {  
    "key": "MiClave"  
  }  
}
```

(Nota: Si existieran más propiedades con defaults, aparecerían aquí automáticamente gracias al merge).

15. Guía de Trabajo para Nuevos Microservicios

Cuando un desarrollador crea un nuevo microservicio para el grabador NXT (por ejemplo, un nuevo gestor de VPN o un servicio de analítica), este servicio necesitará configuración.

Para integrar esta configuración en el sistema centralizado, el desarrollador debe seguir un flujo de trabajo muy específico diseñado para garantizar la coherencia, evitar el código duplicado y eliminar la deuda técnica.

A continuación, se detallan los pasos lógicos y las decisiones de diseño que se deben tomar.

Paso 1: Definición y Generación del Esquema

El primer paso es definir qué propiedades necesita el microservicio, bajo qué nodo colgarán (ej. `config.vpn...`), sus tipos, límites y valores por defecto.

Regla de Oro:  **NUNCA se debe escribir el fichero YAML a mano.**

Si los valores por defecto (`default`), los tamaños máximos (`maxLength`) o los rangos permitidos están definidos como constantes en el código C/C++ del microservicio, escribir un archivo YAML a mano viola el principio de Fuente Única de Verdad (Single Source of Truth). Si en el futuro un desarrollador cambia un puerto por defecto en el código y olvida actualizar el YAML escrito a mano, se producirá un desfase crítico entre lo que la interfaz web valida y lo que el microservicio realmente hace.

La Solución: El propio código del microservicio debe ser capaz de **autogenerar** su propio archivo YAML a partir de sus estructuras de datos internas.

Para lograr esto, el desarrollador puede optar por una de estas dos estrategias:

1. Generación en Tiempo de Compilación / Ejecución en PC:

El microservicio se programa de manera que, al ejecutarse con un flag específico (ej. `./my_service --generate-schema`), imprima el texto YAML por la salida estándar. Durante la fase de desarrollo o en el pipeline de compilación de la imagen Yocto, el binario se ejecuta en el PC del desarrollador, genera el fichero `.yaml` y este se empaqueta automáticamente en la carpeta `/apps/la-system-properties-validator/conf/` de la imagen final.

2. Generación Dinámica vía D-Bus:

Si el microservicio ya implementa una interfaz D-Bus para comunicarse con el sistema, se puede programar un método D-Bus específico (ej. `GetSchemaDefinition()`). El Validator Service o el sistema de arranque puede llamar a este método para que el

microservicio le entregue la cadena de texto YAML con su esquema en tiempo de ejecución.

Ambos métodos garantizan que el código fuente del microservicio sea el único dueño y creador de sus reglas de configuración.

Paso 2: Lectura y Consumo de la Configuración

Una vez que el Validator Service tiene el esquema y el sistema está funcionando, el microservicio necesita leer la configuración que el usuario ha establecido para poder operar.

Existen dos caminos válidos para consumir esta configuración, y la elección dependerá de la complejidad del modelo de datos del microservicio:

Camino A: Lectura de Lista de Propiedades (Para configuraciones sencillas)

Si el microservicio tiene una configuración plana o con pocos parámetros (ej. activar un modo, definir un timeout y un puerto), la forma más ligera y directa es leer las propiedades individuales mediante llamadas D-Bus al servicio de persistencia. Si la propiedad no existe el microservicio pone el valor de defecto que conoce.

Camino B: Lectura del Objeto JSON Completo (Para configuraciones complejas)

Si el microservicio maneja configuraciones anidadas, listas dinámicas (arrays de elementos) o estructuras muy complejas, leer propiedad por propiedad mediante pares clave-valor (ej. `config.vpn.peer.01.ip`, `config.vpn.peer.01.port...`) requiere escribir mucho código tedioso para reconstruir la lógica.

En estos casos, lo ideal es trabajar con el **árbol de configuración completo**.

- **¿Cómo se hace?** El microservicio realiza una petición D-Bus al componente `core-legacy` (que actúa como puente/proxy hacia la lógica de la API REST), solicitándole el JSON completo de su nodo base (ej. Dame el nodo `config.vpn`).
- **El resultado:** El `core-legacy` devuelve un objeto JSON estructurado, jerárquico y totalmente "hidratado" (con todos los valores de defecto insertados y las listas expandidas). El microservicio solo tiene que parsear este JSON utilizando una librería estándar y volcarlo directamente en sus estructuras C++ (`structs` o `classes`).

Paso 3: Reaccionar a los Cambios en Caliente

Finalmente, el microservicio no solo debe leer la configuración al arrancar, sino también reaccionar cuando el usuario la modifica desde la web o la consola.

Para ello, el microservicio debe suscribirse al evento D-Bus global:

```
"CONFIG_CHANGED", "s", path
```

Cuando el microservicio reciba esta señal y compruebe que el `path` modificado pertenece a su rama (ej. empieza por `config.vpn`), deberá volver a ejecutar el **Paso 2** (ya sea leyendo las propiedades afectadas o pidiendo de nuevo el JSON completo a `core-legacy`) y aplicar los cambios internamente de forma dinámica.

Resumen del Flujo para el Desarrollador:

1. Diseña tus estructuras en C/C++.
2. Escribe una función en tu código que genere el esquema en formato YAML.
3. Asegura que ese YAML llegue al Validator (vía compilación u ofreciéndolo por D-Bus).
4. Elige cómo leer tus datos (Propiedades sueltas vs. JSON desde `core-legacy`).
5. Escucha las señales D-Bus para actualizar tu estado en tiempo real.

16. Conclusión

La arquitectura del **Sistema de Gestión de Configuración** representa un avance en la configuración del dispositivo. Al transitar de un modelo de configuración legacy a un sistema basado en modelos formales, hemos establecido una base sólida para la escalabilidad y la fiabilidad a largo plazo.

Este diseño logra tres objetivos estratégicos que transforman el ciclo de desarrollo y mantenimiento:

1. Soberanía del Backend ("Backend-Driven Development")

El **Registro de Propiedades** se convierte en la única fuente de verdad. Las reglas de negocio, validaciones, rangos y estructuras de datos se definen una sola vez, en el código que implementa la funcionalidad.

- *Beneficio:* Eliminamos la lógica de validación duplicada (y a menudo desincronizada) en el frontend. Si el backend cambia un rango de puertos o añade una nueva opción a una lista, el cambio se propaga automáticamente a todas las interfaces sin necesidad de reescribir código en la Web o la CLI.

2. Interfaces Autoadaptables

Tanto la API REST como la CLI son agnósticas respecto al contenido específico que gestionan; actúan como motores de renderizado de los datos y esquemas que reciben.

- *Beneficio:* **Angular renderiza la UI basándose en hechos, no en suposiciones.** Los formularios web se construyen semi dinámicamente a partir de la definición del esquema. Esto reduce drásticamente el tiempo de desarrollo de nuevas pantallas de configuración y elimina una clase entera de bugs de interfaz.

3. Integridad y Seguridad por Defecto

Al imponer un paso de validación obligatorio y centralizado, junto con mecanismos de transaccionalidad (**PUT** atómico) y control de concurrencia (**Locking**), el sistema protege proactivamente la integridad del dispositivo.

- *Beneficio:* El sistema es resistente a errores humanos, condiciones de carrera y ataques básicos. Garantizamos que el dispositivo nunca quede en un estado de configuración inválido o inconsistente.

En resumen, este sistema no es solo una herramienta de almacenamiento de datos; es una **plataforma de gestión de configuración**. Minimiza la deuda técnica, asegura una coherencia total entre las múltiples interfaces de gestión y proporciona una experiencia de usuario robusta y moderna, facilitando enormemente la evolución futura del producto.

*Nota deuda técnica: al evitar soluciones temporales o parches en el frontend para manejar datos incompletos, al resolver los problemas de integridad y estructura en el origen (el backend), se evita escribir código defensivo y repetitivo en la interfaz de usuario, haciendo que el código sea más limpio y fácil de mantener a largo plazo.

17. Edición Masiva: JSON Patch (RFC 6902)

Para escenarios de configuración masiva, realizar actualizaciones de configuración utilizando los métodos tradicionales (`GET` completo -> modificar en local -> `PUT` completo) resulta ineficiente, lento y propenso a errores.

Para solucionar esto, la API de NXT implementa el estándar **JSON Patch (RFC 6902)** en el endpoint `PATCH`, pero potenciado con el motor de comodines y filtrado nativo del sistema.

Este enfoque permite enviar **"instrucciones quirúrgicas"** a los equipos. Le dices al grabador *qué* debe cambiar, sin necesidad de conocer su estado interno completo.

El Reto de la Configuración Masiva

Pongamos el ejemplo de realizar tres tareas en 500 grabadores con diferentes números de cámaras ip licenciadas:

1. Cambiar la IP del servidor central al que se envían las alarmas.
2. Borrar un usuario administrador antiguo que ya no está en la empresa.
3. Activar el envío de metadatos **solo en las cámaras que estén físicamente instaladas**.

Hacer el paso 3 con REST tradicional obligaría al servidor central a consultar primero cada uno de los 500 grabadores para averiguar cuántas cámaras tiene instaladas cada uno, y luego generar 500 JSONs distintos.

La Solución NXT: RFC 6902 + Comodines (`{ }`) + Filtros (`where`)

La implementación acepta un array de operaciones (`op`, `path`, `value`), donde el `path` utiliza la sintaxis estándar con barras `/`.

Esto permite al servidor central enviar **exactamente el mismo payload (JSON) a los 500 equipos**, delegando la lógica de resolución de instancias al propio firmware de cada grabador.

Ejemplo Práctico de Payload Masivo

Petición HTTP:

PATCH /config

Content-Type: application/json-patch+json

```
[
  {
    "op": "replace",
    "path": "/config/alarms/server/ip",
    "value": "10.100.50.25"
  },
  {
    "op": "remove",
    "path": "/config/device/users/04"
  },
  {
    "op": "add",
    "path": "/config/network/snmp/v3",
    "value": {
      "enabled": true,
      "username": "admin_snmp",
      "authProtocol": "SHA",
      "privProtocol": "AES"
    }
  },
  {
    "op": "replace",
    "path": "/config/video/cam/{}/media/metadata/enabled",
    "value": true,
    "where": "eq(base.installed|true)"
  }
]
```

Beneficios Arquitectónicos

- **Ahorro de Ancho de Banda:** Los payloads ocupan apenas unos bytes, en contraste con el tamaño que supondría enviar configuraciones completas.

- **Idempotencia y Orquestación:** Herramientas como el CS+ pueden enviar este JSON a toda la flota simultáneamente.
- **Transaccionalidad (Atomicidad):** O se aplican todas las operaciones del array, o no se aplica ninguna. Si el `add` del SNMP falla la validación del esquema (ej. el protocolo no existe), se aborta el cambio de IP de alarmas, garantizando que el equipo nunca quede en un estado híbrido o inconsistente.

Expansión Multinivel y Filtrado Posicional

Los modelos de datos en sistemas complejos a menudo contienen colecciones anidadas dentro de otras colecciones. Por ejemplo, un grabador tiene múltiples *cámaras*, y cada cámara tiene múltiples *streams de vídeo*.

Si un sistema de configuración masiva necesita modificar una propiedad profunda, la ruta del JSON Patch contendrá varios comodines, por ejemplo:

```
path: "/config/video/cam/{}/stream/{}/resolution"
```

El problema: Si utilizamos un `where` con un simple string (ej. `"where":`

`"eq(enabled|true) "`), el sistema se enfrentaría a una ambigüedad insalvable: ¿El filtro debe evaluar si la *cámara* está habilitada, o si el *stream* está habilitado?

La solución: Filtros Posicionales mediante Arrays.

Para resolver la expansión multinivel, la propiedad `where` del JSON Patch soporta recibir un **Array de Strings**. El backend mapea este array de izquierda a derecha contra los comodines `{}` encontrados en el `path`.

- El elemento en la posición `[0]` se evalúa en el contexto del primer comodín (ej. la cámara).
- El elemento en la posición `[1]` se evalúa en el contexto del segundo comodín (ej. el stream).
- Si no se desea aplicar ningún filtro en un nivel intermedio, se puede enviar un string vacío `""` o `null`. Si se envía un string simple en lugar de un array, el sistema asume retrocompatibilidad y lo aplica únicamente al primer nivel.

Ejemplo Práctico: Filtrado en Dos Niveles

Imagina que deseamos reconfigurar la resolución a "1080p" de forma masiva, pero con una condición de negocio muy estricta: **Solo debe aplicarse en las cámaras que estén físicamente instaladas, y dentro de estas, solo en los streams que estén configurados con el códec H.264.**

```
PATCH /device-service/properties/data
Content-Type: application/json-patch+json
```

```
[
  {
    "op": "replace",
    "path": "/config/video/cam/{}/media/video/{}/resolution",
    "value": "1080p",
    "where": [
      "eq(base.installed|true)",
      "eq(codec|H264)"
    ]
  }
]
```

¿Cómo procesa el backend este escenario multinivel?

1. Análisis del Nivel 1 (Cámaras):

El motor recursivo llega al primer {} (/config/video/cam/{}/...). Itera sobre todas las cámaras existentes en la memoria del grabador y les aplica el primer filtro del array: `eq(base.installed|true)`.

Supongamos que detecta que las cámaras 000 y 005 están instaladas. Las cámaras restantes son ignoradas.

2. Análisis del Nivel 2 (Streams):

Para cada cámara validada (la 000 y la 005), el motor desciende hasta el segundo {} (.../media/video/{}/resolution). Ahora, el contexto de evaluación cambia. Se itera sobre los streams locales de esa cámara específica y se aplica el segundo filtro del array: `eq(codec|H264)`.

- En la cámara 000, el stream 0 es H264 y el stream 1 es H265. Pasa el stream 0.
- En la cámara 005, los streams 0 y 1 son H264. Pasan ambos.

3. Generación de Operaciones Atómicas:

Una vez concluida la recursividad, el sistema genera dinámicamente las rutas absolutas:

- `config.video.cam.000.media.video.0.resolution = "1080p"`
- `config.video.cam.005.media.video.0.resolution = "1080p"`
- `config.video.cam.005.media.video.1.resolution = "1080p"`

Estas rutas se aplanan, se validan contra el esquema central y se escriben de forma atómica.

Valor Estratégico: Gracias al filtrado posicional, el servidor central no necesita mantener una réplica exacta del estado interno de los 500 grabadores, ni realizar bucles de consultas previas. Envía **intenciones de negocio declarativas** y confía en que el motor de configuración del dispositivo (NXT) las aplique de forma quirúrgica en el contexto correcto.

15.1 banesto.mvc

```
START
ADD users user name "banesto" password "banesto"
END
```

```
[
  {
    "op": "add",
    "path": "/config/device/users/user",
    "value": {
      "name": "banesto",
      "password": "banesto"
    }
  }
]
```

15.2 banesto2.mvc

```
START
users user name "banesto" profile=advanced
END
```

```
[
  {
    "op": "replace",
    "path": "/config/device/users/user/{}/profile",
    "value": "advanced",
    "where": "eq(name|banesto)"
  }
]
```

15.2 bankia_test.mvc

START

FILTER ethernet ip address 172.16.90.73

video camera 1 description="103 B3-PB-ESP.V"

video camera 2 description="104 B3-PB-PAS V"

video camera 3 description="105 B3-PB-PASSA"

video camera 4 description="106 B3-LOC36-43"

END

```
[
  {
    "op": "if",
    "path": "/config/network/if/eth0/ip-user.{}",
    "match": "eq(addr|172.16.90.73)",
    "then": [
      {
        "op": "replace",
        "path": "/config/video/cam/000/base/name",
        "value": "103 B3-PB-ESP.V",
      },
      {
        "op": "replace",
        "path": "/config/video/cam/001/base/name",
        "value": "104 B3-PB-PAS V",
      },
      {
        "op": "replace",
        "path": "/config/video/cam/002/base/name",
        "value": "105 B3-PB-PASSA",
      },
      {
        "op": "replace",
        "path": "/config/video/cam/003/base/name",
        "value": "106 B3-LOC36-43",
      }
    ]
  }
]
```

ANEXO 1. Estudio de soluciones open source

Antes de empezar el desarrollo se realizó un previo estudio de opciones open source. Vemos que existen piezas sueltas, pero tendríamos que pegarlas nosotros mismos, escribiendo tanto o más código "pegamento" que el que hemos escrito para nuestro motor, y con un resultado probablemente menos eficiente y más difícil de depurar.

Análisis de Alternativas (y por qué nuestro motor gana)

1. gRPC / Protobuf / Thrift:

- *Lo que son:* Frameworks para definir servicios y mensajes tipados, con generación de código.
- *El problema:* Son excelentes para la transmisión, pero **no gestionan el estado ni la validación de negocio** de una configuración jerárquica. Tendríamos que escribir igualmente toda la lógica de "guardar en DB", "validar rangos", "gestionar defaults" y "fusionar". Además, son pesados para sistemas embebidos pequeños.

2. Yang / NETCONF / RESTCONF:

- *Lo que son:* Estándares de la industria de telecomunicaciones para configuración de dispositivos de red.
- *El problema:* Son **extremadamente complejos** y pesados. Implementar un servidor NETCONF completo requiere meses de trabajo o comprar pilas comerciales muy caras (como ConfD). Para un NVR o dispositivo IoT, es matar moscas a cañonazos.

3. Librerías de Configuración (libconfig, jsoncpp, yaml-cpp):

- *Lo que son:* Solo leen/escriben ficheros.
- *El problema:* No te dan validación centralizada, ni API REST, ni CLI, ni gestión de concurrencia (lock). Hemos construido todo eso *encima* de una lógica simple de clave-valor.

4. Key-Value Stores (Redis, ETCD):

- *Lo que son:* Bases de datos.

- *El problema:* Son solo almacenamiento. No validan que "el puerto debe ser < 65535". No tienen concepto de esquema.

Argumentos de Defensa para nuestro Motor

1. **Integración Vertical:** nuestro motor une **Almacenamiento + Validación + API REST + CLI + Seguridad**. Usar librerías externas para cada cosa resultaría en un "Frankenstein" de dependencias difícil de mantener y actualizar en un entorno Yocto.
2. **Adaptación al Dominio:** nuestro sistema entiende conceptos nativos de nuestro negocio: "ResourceFlag", "Mask", "Password Policy". Una librería genérica no.
3. **Ligereza:** nuestra solución es C++ puro, optimizado, sin máquinas virtuales ni runtimes pesados. Ideal para embebidos.
4. **Control Total:** Si mañana necesitamos cambiar cómo se encriptan los passwords o añadir un nuevo tipo de transporte, podemos hacerlo rápidamente. Con un framework de terceros, podríamos estar bloqueado o tener que esperar meses.
5. **Tamaño del Código:** Lo que hemos escrito no es "grande". Son unos pocos miles de líneas de código muy focalizadas. Importar librerías y frameworks añadiría megabytes de código binario y complejidad de compilación.

Conclusión:

No hemos caído en el "Not Invented Here". Estamos aplicando la ingeniería **adecuada al problema**. Se ha construido un middleware especializado que aporta un valor (coherencia, validación, interfaz unificada) con un coste de mantenimiento muy bajo comparado con integrar y pelear con frameworks empresariales gigantescos.

ANEXO 2: Comparativa con GraphQL

Lo más parecido a lo que hemos implementado es GraphQL pero con las ventajas que sigue siendo REST. Por tanto, juntando lo mejor de los dos mundos.

Compararemos finalmente **GraphQL** con nuestro sistema **REST++** (evolución REST tradicional).

A diferencia de las APIs REST tradicionales, donde el servidor define qué datos recibes, con GraphQL **tú (el cliente) decides exactamente qué necesitas** y nada más. Fue creado por Facebook en 2012 y lanzado como código abierto en 2015.

¿En qué se diferencia de REST?

Imagina que quieres obtener el nombre de un usuario y los títulos de sus últimos 3 posts.

Con REST:

Tendrías que hacer múltiples peticiones a diferentes "endpoints":

1. `/usuarios/1` (para el nombre).
2. `/usuarios/1/posts` (para todos los posts, aunque solo quieras los títulos). *Resultado:* Recibes demasiada información (**over-fetching**) o tienes que hacer muchas llamadas (**under-fetching**).

Con GraphQL:

Haces **una sola petición** enviando un esquema de lo que quieres:

GraphQL

```
None
{
  usuario(id: "1") {
    nombre
    posts(last: 3) {
      titulo
    }
  }
}
```

Resultado: El servidor te responde exactamente con ese JSON. Ni más, ni menos.

Conceptos Clave

Para entender cómo funciona "bajo el capó", hay tres pilares:

- **Query (Consulta):** Es lo que usas para leer datos (equivalente al **GET** en REST).
- **Mutation (Mutación):** Se usa para crear, actualizar o borrar datos (equivalente a **POST**, **PUT** o **DELETE**).
- **Schema (Esquema):** Es el contrato entre el cliente y el servidor. Define qué tipos de datos existen y cómo se relacionan.

Ventajas y Desventajas graphql

Ventajas	Desventajas
Adiós al over-fetching: Solo descargas lo que usas (ideal para móviles).	Curva de aprendizaje: Es más complejo de configurar inicialmente que REST.
Tipado fuerte: Gracias al esquema, sabes exactamente qué esperar.	Caché compleja: Al usar siempre el mismo endpoint (/graphql), la caché HTTP estándar no funciona tan fácil.
Evolución sin versiones: No necesitas un /v1/ o /v2/ ; solo añades campos nuevos.	Consultas pesadas: Un cliente podría pedir datos muy anidados y saturar el servidor.

Comparativa de nuestra solución con GraphQL

1. Selección de Campos (Data Fetching)

- **GraphQL:** Permite al cliente pedir exactamente lo que necesita:

```
query { camera { connection { ip } } }
```

- **NXT:** Implementado esto con `fields=...`:
`/config.video.cam?fields=base.ipcamera.connection|base.ipcamera.account`

Veredicto: GraphQL es más flexible nativamente (puedes pedir campos disjuntos fácilmente), pero la implementación de `fields` con filtrado profundo cubre el 95% de los casos de uso de optimización de ancho de banda. ¡Empate técnico para el caso de uso!

2. Introspección (Schema)

- **GraphQL:** Tiene un sistema de tipos fuerte y una API de introspección (`__schema`) que permite a herramientas como GraphQL autocompletar y validar consultas.
- **NXT:** propio sistema de introspección:
`GET /schema?node=...`
Devuelve un JSON que describe tipos, rangos, defaults...
- **Veredicto:** El sistema NXT es **más rico semánticamente** para configuración. GraphQL te dice "esto es un String", pero el esquema de NXT te dice "es un String, máx 32 chars, requiere reinicio, y si es password tiene política nivel 2". Superado a GraphQL estándar en metadatos de negocio.

3. Evitar Over-fetching y Under-fetching

- **GraphQL:** Resuelve el problema de pedir demasiados datos o tener que hacer muchas peticiones.
- **NXT:**
 - **Over-fetching:** Resuelto con `fields`.
 - **Under-fetching:** Resuelto con la lógica de **Merge**. Al devolver el objeto completo con defaults, evitas que el cliente tenga que hacer lógica extra o pedir configuraciones base por separado.

4. Mutaciones (Escritura)

- **GraphQL:** Usa `Mutation`. Es rpc (Remote Procedure Call) disfrazado.

`mutation { updateCamera(id: "01", ip: "...") { success } }`
- **NXT:** Usa REST puro (`PUT`, `POST`, `DELETE`, `PATCH`).
- **Veredicto:** REST es mucho más estándar para el cacheo HTTP y la simplicidad. La implementación NXT transaccional del `PUT` (validar todo antes de escribir nada) es tan robusta como cualquier mutación bien hecha.

Conclusión:

En NXT se ha construido un sistema **REST++**.

Se ha cogido la simplicidad y compatibilidad de REST y se le ha inyectado la inteligencia (Esquema, Selección de Campos, Validación Fuerte) que normalmente hace que la gente se pase a GraphQL.

Para un sistema embebido/IoT, **el enfoque NXT es superior a GraphQL** porque:

1. **Menos Overhead:** No necesita un motor de query complejo en el dispositivo.
2. **Cacheabilidad:** Usa HTTP estándar. Al usar nodos (URLs únicas) y verbos estándar, cualquier CDN o Proxy (como Varnish o Nginx) puede cachear tus respuestas sin configuración extra. GraphQL no puede hacer esto fácilmente.
3. **Metadatos Específicos:** El esquema habla el lenguaje del hardware (recursos fijos, reinicios), algo que en GraphQL se tendría que "inventar" con directivas custom..
4. **Menos "Boilerplate" en el Cliente:** En GraphQL, el cliente tiene que definir la consulta. En tu sistema, el cliente solo pide el recurso y el servidor (que conoce el esquema) le da todo lo que necesita.
 - a. En un grabador, el frontend suele ser una serie de formularios.
 - b. **En GraphQL:** El frontend tiene que saber exactamente qué pedir. Si añades una nueva función a la cámara, tienes que actualizar la consulta en el cliente.
 - c. **En nuestro sistema:** El cliente pide el nodo y recibe el objeto completo (incluyendo lo nuevo por defecto). El frontend puede ser **genérico**: si llega una clave nueva, dibuja un campo nuevo. Es un sistema orientado a datos, no a consultas.
5. **Ahorro de Almacenamiento:** La estrategia de "solo guardar lo diferente" no la tiene GraphQL.
6. **Inmutabilidad y Valores de Fábrica:** Al guardar solo lo "diferente al defecto", hemos implementado un sistema de **"Factory Reset" instantáneo**. Si quieres volver a los valores de fábrica, solo tenemos que borrar los deltas del almacén clave-valor. El sistema seguirá funcionando con los valores por defecto que el servidor ya conoce.
7. **Eficiencia en Memoria Flash:** Los videograbadores suelen usar memorias flash (NAND/NOR) que tienen ciclos de escritura limitados. Al aplanar y guardar solo lo diferente, minimizas el desgaste del hardware (*wear leveling*) y el tamaño de los archivos de configuración. En GraphQL, los valores por defecto suelen estar enterrados en el código o en la lógica del resolver. En nuestro sistema, al **guardar solo el delta**, tienes una separación física entre:
 - a. **Lo que el fabricante decidió** (Valores por defecto).
 - b. **Lo que el usuario cambió** (Deltas). Esto hace que la recuperación ante errores sea trivial: si un delta corrompe el sistema, lo borras y el valor por defecto "emerge" automáticamente.
8. **Consistencia Atómica:** Al usar **PUT** para nodos enteros y validar contra el esquema antes de escribir, aseguras que el grabador nunca quede en un estado inconsistente (por ejemplo, una resolución de video que no soporta el bitrate asignado).

Conclusión Final

Si bien hay muchas maneras de hacer las cosas la escogida salvaguarda muchas de las cosas buenas del HM pero modernizadas a un sistema distribuido LINUX. Hemos pasado de registrar con CMD a un sistema moderno que permite el desarrollo de microservicios independientes.

GraphQL es excelente para redes sociales o e-commerce donde las relaciones entre datos son caóticas. Pero para un NVR, donde la jerarquía es clara (Cámara -> Stream -> Resolución), **nuestro sistema es mejor**. Es más fácil de depurar (puedes usar un simple navegador o `curl` para ver cada nodo), es más ligero para la CPU del grabador y respeta mejor la semántica de estado con `PUT/PATCH`.

Documentación técnica: ONVIF Profile G en NXT

1. Visión General

Introducción

En este documento explicaremos cómo la implementación ONVIF realizada en el grabador NXT maneja el sistema de propiedades, y cómo su ciclo de vida se compara con las otras dos interfaces de configuración principales: la API REST y la CLI (Consola).

El Perfil G de ONVIF es la especificación estándar para la búsqueda, reproducción y recuperación de medios grabados. La implementación de este perfil en el grabador NXT —actuando como Dispositivo (Servidor)— garantiza que plataformas de terceros (clientes ONVIF, sistemas VMS o centros de control) puedan acceder de forma estandarizada e interoperable a las grabaciones almacenadas en sus discos duros.

Objetivo Principal

Proporcionar una interfaz estandarizada para que clientes externos puedan consultar el inventario de grabaciones del NXT, buscar fragmentos específicos de vídeo/audio/metadatos y solicitar su reproducción o descarga, sin depender de protocolos propietarios.

Arquitectura de Roles

Para esta implementación, la arquitectura se define de la siguiente manera:

- **Dispositivo Profile G (El Grabador NXT / Servidor):** Es el sistema que almacena los flujos de vídeo grabados, aloja la base de datos de eventos/tiempos y expone los servicios de ONVIF para responder a las consultas de reproducción.
- **Cliente Profile G (VMS de terceros / Visores):** Es el software o sistema externo que se conecta al NXT para solicitar el vídeo grabado.

Capacidades y Servicios Implementados (Features)

- **Servicio de Búsqueda (Search Service):** El grabador NXT es capaz de recibir y procesar consultas de clientes externos sobre el contenido almacenado.
 - **Búsqueda por Tiempo:** El NXT devuelve las pistas (*tracks*) disponibles respondiendo a peticiones de rangos de fecha y hora.

- **Búsqueda por Eventos:** Permite a los clientes localizar grabaciones asociadas a eventos específicos (por ejemplo, alarmas, detección de movimiento o analíticas previamente registradas por el NXT).
- **Servicio de Reproducción (Replay Service):** El NXT proporciona una URI de RTSP (*Real-Time Streaming Protocol*) específica para que el cliente acceda al flujo de vídeo grabado.
 - **Control de flujo RTSP:** El grabador procesa comandos estándar de reproducción enviados por el cliente, soportando acciones como: *Play* (reproducir desde un punto temporal exacto), *Pause* (pausar), y ajuste de velocidad (*Scale*) para cámara rápida o cámara lenta.
- **Servicio de Control de Grabación (Recording Control Service):** El grabador expone la estructura lógica de su almacenamiento interno (*Tracks* y *Recording Jobs*). Permite a los clientes autorizados consultar la configuración de las pistas que se están grabando y conocer su estado actual.
- **Entrega de Metadatos y Audio (Soporte Multitrack):** Además del flujo de vídeo, la implementación permite que el NXT envíe pistas de audio grabado y metadatos asociados, manteniendo la sincronización original durante la reproducción solicitada.

Beneficios de la Implementación en el Grabador NXT

Al integrar el Perfil G como servidor, el grabador NXT aporta un gran valor comercial y operativo:

- **Apertura y Compatibilidad Total (Agnóstico):** Permite que el NXT se integre de manera transparente con cualquier VMS del mercado (como Milestone, Genetec, ExacqVision, etc.) sin necesidad de desarrollar APIs propietarias o plugins de integración complejos.
- **Arquitectura Distribuida:** Facilita que el NXT funcione como un nodo de almacenamiento local en arquitecturas multisitio, donde un centro de control remoto puede extraer vídeo post-evento solo cuando es necesario, optimizando el ancho de banda de red.
- **Robustez en la Recuperación:** Proporciona un método fiable y estandarizado para que sistemas de nivel superior auditen y recuperen evidencias en vídeo directamente desde los discos del NXT.

2. Modelo de Lectura de Configuración (Readers)

La lectura de los datos configurables se realiza con un ciclo de vida diferente dependiendo de la interfaz de acceso:

- **API REST & CLI:** Son clientes "**Sin Estado**" (**Stateless**). Cuando se realiza una consulta, solicitan directamente los nodos al gestor central del sistema de propiedades y componen las vistas en tiempo real gracias a la hidratación con el esquema de validación.
- **ONVIF (Core Legacy):** Es un cliente "**Con Estado**" (**Stateful**). Al arrancar el equipo, el servicio *core-legacy* lee la configuración desde la persistencia y puebla sus propias estructuras internas en C/C++. Si ocurren cambios de configuración orquestados desde REST o CLI, el servicio reacciona a los eventos D-Bus en tiempo real, manteniendo sus estructuras de memoria perfectamente sincronizadas. Sobre estas estructuras opera el motor de serialización y comunicación SOAP de ONVIF.

3. Modelo de Escritura de Configuración (Writers)

De igual modo, el flujo de escritura difiere estructuralmente según su origen, aunque todos convergen en la misma fuente única de la verdad.

- **API REST & CLI:** Solicitan nodos o vistas de las propiedades y, una vez modificadas por el usuario, las escriben **directamente** en el sistema de propiedades (previa validación estricta contra el esquema). Realizada la escritura, se emiten eventos D-Bus para que los servicios en segundo plano actualicen sus estructuras internas.
- **ONVIF:** Cuando se ejecuta una petición ONVIF (ej. un VMS cambia la resolución de una cámara), el flujo es **inverso**. Primero se actualizan las estructuras internas de memoria de C/C++ y, a continuación, se activa el motor de serialización para "volcar" esos cambios en el sistema de propiedades oficial, donde igualmente son validados contra el esquema antes de persistirse.
- **Driver de Cámara IP:** Como caso excepcional de automatización, el propio motor de vídeo puede auto-configurar el usuario y el *password* al que se conecta a una cámara ip si, tras leer las credenciales, decide aplicar un cambio de seguridad (ej. porque se ha forzado que todas las cámaras compartan contraseña o porque la cámara aún tiene credenciales de fábrica).

4. Optimizador de Escritura ONVIF (Dirty Tracking)

Dado que las peticiones ONVIF suelen llegar en ráfagas asíncronas (ej. un VMS configurando 20 parámetros de una cámara consecutivamente), el sistema de escritura ONVIF implementa un patrón de **Integración y Seguimiento de Suciedad (Dirty Tracking)** para evitar cuellos de botella y desgaste en la memoria Flash.

El Flujo del Tracker:

1. Cuando un método ONVIF modifica una configuración (ej. añade una IP, crea un usuario o cambia una resolución), el sistema no graba inmediatamente. En su lugar, **marca la estructura lógica afectada como "sucia"** en un mapa en memoria y activa un temporizador (Timer) de varios segundos.
2. Si llegan nuevas peticiones, simplemente se añaden nuevas marcas al mapa y se reinicia el temporizador (efecto *Debounce*).
3. Cuando el temporizador expira de forma definitiva, se encola un mensaje al hilo dedicado a la persistencia (Task Thread).
4. Este hilo consume el mapa de "marcas sucias" y lanza un proceso de serialización **altamente granular**. En lugar de reescribir toda la configuración del dispositivo, el algoritmo navega directamente a la sub-estructura específica (ej. Cámara 5, Stream 2) y actualiza en D-Bus exclusivamente los pares clave-valor que han mutado, previa validación.

Niveles de Granularidad Soportados

La implementación actual del grabador NXT permite resoluciones de guardado extremadamente precisas mediante el paso de etiquetas y valores (`std::map<std::string, int>`). Algunos ejemplos del registro de marcas del Tracker:

- **Configuraciones Globales (Sin índices):**
 - `g_properties_dirty_tracker.mark_dirty("CFG_MAIN", {});`
 - `g_properties_dirty_tracker.mark_dirty("CFG_PORS", {});`
 - `g_properties_dirty_tracker.mark_dirty("CFG_NTP", {});`
 - `g_properties_dirty_tracker.mark_dirty("CFG_CLOCK", {});`
 - `g_properties_dirty_tracker.mark_dirty("CFG_KEYSTORE", {});`
- **Configuraciones Modulares (Con índices de 1 nivel):**
 - `g_properties_dirty_tracker.mark_dirty("CFG_ETHERNET",
{{"interface_idx", 0}});`
 - `g_properties_dirty_tracker.mark_dirty("CFG_OUTPUTS",
{{"output_idx", iOutputIdx}});`

- `g_properties_dirty_tracker.mark_dirty("CFG_USERS", {"user_idx", i});`
- `g_properties_dirty_tracker.mark_dirty("CFG_USERS", {"super_user", -1});`
- `g_properties_dirty_tracker.mark_dirty("CFG_SWITCH", {"port_idx", i});`

- **Configuración de Vídeo (Alta complejidad multinivel):**

- `g_properties_dirty_tracker.mark_dirty("CFG_VIDEO", {"camera_idx", iCameraIdx}, {"connection", -1});` *(Todo el bloque de conexión de una cámara)*
- `g_properties_dirty_tracker.mark_dirty("CFG_VIDEO", {"camera_idx", iCameraIdx}, {"account", -1});` *(usuario y password de una cámara)*
- `g_properties_dirty_tracker.mark_dirty("CFG_VIDEO", {"camera_idx", iCameraIdx}, {"capabilities", -1});` *(flag de capacidades de una cámara)*
- `g_properties_dirty_tracker.mark_dirty("CFG_VIDEO", {"camera_idx", iCameraIdx}, {"lock", -1});` *(mac lock y mac resources de una cámara)*
- `g_properties_dirty_tracker.mark_dirty("CFG_VIDEO", {"camera_idx", iCameraIdx}, {"video_sources", -1});` *(multi canal)*
- `g_properties_dirty_tracker.mark_dirty("CFG_VIDEO", {"camera_idx", iCameraIdx}, {"imaging", -1});` *(image settings)*
- `g_properties_dirty_tracker.mark_dirty("CFG_VIDEO", {"camera_idx", uiCameraIdx}, {"stream_idx", uiStreamIdx});` *(Un único stream específico)*
- `g_properties_dirty_tracker.mark_dirty("CFG_VIDEO", {"camera_idx", uiCameraIdx}, {"recorder_idx", uiStreamIdx});` *(Un job de grabación específico de una cámara)*
- `g_properties_dirty_tracker.mark_dirty("CFG_VIDEO", {"camera_idx", iCameraIdx}, {"preset_idx", iPresetIdx});` *(Un preset PTZ específico)*

- `g_properties_dirty_tracker.mark_dirty("CFG_VIDEO",
{{"camera_idx", iCameraIdx}, {"preset_round_list", -1}});`
(Una ronda de presets PTZ)
- `g_properties_dirty_tracker.mark_dirty("CFG_VIDEO",
{{"camera_idx", iCameraIdx}, {"preset_tour", -1}});` *(Un tour PTZ)*
- `g_properties_dirty_tracker.mark_dirty("CFG_VIDEO",
{{"profile_idx", i}});` *(Un perfil ONVIF)*
- `g_properties_dirty_tracker.mark_dirty("CFG_VIDEO",
{{"recording_job_idx", i}});` *(Un recording JOB de ONVIF)*
- `g_properties_dirty_tracker.mark_dirty("CFG_ETHERNET",
{{"interface_idx", i}, {"dhcp_capabilities", -1}});`
(capabilities dhcp)
- `g_properties_dirty_tracker.mark_dirty("CFG_VIDEO",
{{"camera_idx", Source_ptr->bLinkId}, {"all", -1}});` *(toda una cámara, por ejemplo una hija de multicanal).*

Esta arquitectura garantiza que solamente se escriban las propiedades afectadas..

Documentación Técnica: Net-SNMP (snmpd) en NXT

Esta documentación detalla el funcionamiento, configuración y extensión del servicio **snmpd** (Net-SNMP) en un entorno Linux (YOCTO), basándose en la experiencia de integración y resolución de problemas proporcionada.

1. Introducción y Arquitectura

Net-SNMP es una suite de software que implementa el protocolo SNMP (Simple Network Management Protocol) v1, v2c y v3. En Linux, su componente principal es el demonio o agente.

- **snmpd (Agente):** Es el servicio que se ejecuta en segundo plano en el servidor. Escucha peticiones SNMP (GET, GETNEXT, SET) en el puerto UDP 161 y responde con información del sistema o de aplicaciones específicas. También envía alertas (Traps/Informs) al puerto 162.
- **snmp (Cliente/Herramientas):** Comandos de consola como `snmpwalk`, `snmpget`, `snmptranslate` que actúan como "Manager" para consultar al agente.
- **MIBs (Management Information Bases):** Archivos de texto que describen la estructura de los datos (el árbol de OIDs) que el agente puede servir.

2. Instalación de Componentes

Para un entorno de desarrollo e implementación completo, se requieren tres paquetes principales:

None

```
udo apt install snmpd snmp libsnmp-dev
```

- **snmpd:** El servidor/agente.
- **snmp:** Las herramientas de cliente para probar el servidor.
- **libsnmp-dev:** Librerías de desarrollo C/C++. Son **indispensables** para compilar subagentes y utilizar herramientas de generación de código como `mib2c`.

Validación de la instalación:

None

```
net-snmp-config --version # Ejemplo: 5.8
```

```
systemctl status snmpd # Verifica que el servicio corre
```

3. Configuración del Agente y del Cliente

Es crucial distinguir entre los dos ficheros de configuración principales, ya que sirven para propósitos opuestos.

A. Fichero del Agente: `/etc/snmp/snmpd.conf`

Este fichero controla **cómo se comporta el servidor snmpd**. Define seguridad, acceso y puertos.

Parámetros clave configurados:

1. `agentaddress udp:161`:
 - Por defecto, `snmpd` suele escuchar solo en `localhost` (127.0.0.1) por seguridad.
 - Al cambiarlo a `udp:161`, indicamos que escuche en **todas las interfaces de red** disponibles en el puerto estándar SNMP. Esto permite que sistemas externos (Managers) consulten este equipo.
2. `view systemonly included .1.3.6.1.4.1.41289`:
 - Esto es parte del control de acceso (VACM).
 - Define una "vista" (view) llamada `systemonly`.
 - `included` significa que permitimos el acceso a esa rama del árbol OID.
 - `.1.3.6.1.4.1.41289`: Es el OID base (Enterprise OID) de **Lanaccess/ONSAFEHM**. Sin esta línea, aunque el agente tenga los datos, denegaría el acceso a consultarlos por seguridad.

B. Fichero del Cliente/Librería: `/etc/snmp/snmp.conf`

Este fichero controla **cómo se comportan las herramientas** (como `snmpwalk`, `mib2c` o librerías que consumen SNMP). No afecta al demonio servidor directamente, sino a cómo se interpretan y cargan las MIBs.

Parámetros clave configurados:

1. (Comentar esta línea):

- En muchas distribuciones (Debian/Ubuntu), esta línea existe por defecto para restringir la carga automática de MIBs debido a licencias.
- **Acción:** Se debe comentar (`#mibs :`) para permitir que las herramientas carguen **todas** las MIBs disponibles en los directorios configurados.

2. `mibdirs`:

- Define la ruta de búsqueda de los ficheros MIB.
- Configuración usada:
`/usr/share/snmp/mibs:/usr/share/snmp/mibs/iana:/usr/share/snmp/mibs/ietf:/home/xavier/mibs`
- Esto instruye al sistema a buscar tanto en las rutas del sistema como en el directorio personal donde se alojan las MIBs privadas (`/home/xavier/mibs`).

4. Gestión de MIBs y Solución de Problemas Comunes

El problema de las MIBs estándar faltantes

Al ejecutar comandos como `snmpwalk`, es común ver errores como `Cannot find module (SNMPv2-MIB)`. Esto ocurre porque Linux, por temas de licencias, no instala las MIBs propietarias del IETF por defecto.

Solución:

1. Instalar el descargador: `sudo apt-get install snmp-mibs-downloader`
2. Ejecutar la descarga: `sudo download-mibs`
3. Habilitar la carga en la configuración (visto en el punto 3B): `sudo sed -i "s/^\(mibs *: \) .*/#\1/" /etc/snmp/snmp.conf`
4. Reiniciar el servicio para aplicar cambios: `sudo service snmpd restart`

Integración de una MIB Privada (ONSAFEHM)

Para que el sistema entienda nuestros propios objetos (Lanaccess), debemos "compilar" o instalar nuestra MIB.

1. **Ubicación:** Copiar el fichero (ej. `ONSAFEHM-MIB.txt`) a `/usr/share/snmp/mibs`.

2. **Sintaxis Correcta:** El fichero MIB debe empezar estrictamente con `NOMBRE-MIB DEFINITIONS ::= BEGIN.`
3. **Correcciones de Dependencias:**
 - Para usar herramientas como `mib2c`, la MIB debe ser válida y completa.
 - Se añadió la sección `IMPORTS` para definir de dónde vienen tipos básicos como `OBJECT-TYPE`, `TRAP-TYPE` y `enterprises`.
 - Se definió `MODULE-IDENTITY` para `lanaccess`. Esto es obligatorio para que Net-SNMP trate el nodo como un módulo moderno y permita generar código C a partir de él.

Validación del árbol:

El comando `snmptranslate` es vital para verificar que la MIB está bien formada y cargada:

None

```
sudo snmptranslate -M+. -mONSAFEHM -Tp 1.3.6.1.4.1.41289
```

-
- `-mONSAFEHM`: Fuerza la carga del módulo `ONSAFEHM`.
- `-Tp`: Imprime el árbol de forma gráfica.
- Si sale el árbol correctamente dibujado, la MIB está perfecta.

5. Integración con Systemd

Net-SNMP se ejecuta como un servicio del sistema.

- **Estado:** `systemctl status snmpd`
- **Reinicio:** `sudo service snmpd restart` (Necesario cada vez que cambiamos `snmpd.conf` o añadimos binarios de subagentes si están integrados dinámicamente).
- **Logs:** Si hay errores de arranque, suelen verse en `journalctl -xe` o en `/var/log/syslog`.

6. Generación de Código C/C++ (Stubs)

Para que el agente responda a los OIDs definidos en la MIB privada (Lanaccess), necesitamos escribir código C++ que se enlace con el agente. La herramienta `mib2c` automatiza esto convirtiendo definiciones MIB en plantillas de código ("stubs").

Prerrequisitos:

Instalar soporte de Perl para SNMP:

None

```
sudo apt-get install libnet-snmp-perl libsnmp-perl
```

A. Generación de Stubs para Escalares

Los escalares son variables simples (enteros, strings).

- **Comando:** `mib2c -c mib2c.scalar.conf ONSAFEHM:lanaccess`
- **Resultado:** Genera ficheros `.c` y `.h`.
- **Adaptación:**
 1. Renombrar a `.cpp` para usar C++.
 2. Implementar la lógica dentro de las funciones `handle_xxx` generadas para leer el valor real del sistema.
-

B. Generación de Stubs para Tablas

Las tablas son matrices de datos.

- **Comando:** `mib2c -c mib2c.table_data.conf ONSAFEHM:lanaccess`
- **Opción recomendada:** Elegir opción "2) Hold the data for the table solely within the agent" si los datos no cambian muy rápido o si queremos que el agente cachee los datos.
- **Nota de desarrollo:** Al pasar a C++, hay que tener cuidado con conflictos de nombres en las `struct` generadas. El prefijo `m_` es una buena práctica para miembros de estructuras y evitar colisiones con macros de la librería.

C. Generación de Stubs para Traps (Notificaciones)

Las traps son eventos enviados desde el agente al manager (iniciados por el agente).

- **Comando:** `mib2c -c mib2c.notify.conf ONSAFEHM:lanaccess`
- **Resultado:** Genera funciones C que, al ser llamadas por tu aplicación, envían la alerta SNMP a la red.

D. Compilación

El código generado (`.c/.cpp`) debe compilarse utilizando las flags que indica Net-SNMP. Se puede generar un `Makefile` base con:

None

```
mib2c -c mfd-makefile.m2m ONSAFEHM:lanaccess
```

O simplemente usar `net-snmp-config --cflags --libs` para obtener las opciones de compilación necesarias para `g++`.

Resumen del Flujo de Trabajo

1. **Definir MIB:** Crear/Editar `ONSAFEHM-MIB.txt`.
2. **Instalar MIB:** Copiar a `/usr/share/snmp/mibs`.
3. **Verificar:** Usar `snmptranslate` para asegurar que no hay errores de sintaxis.
4. **Generar Código:** Usar `mib2c` para crear los esqueletos C/C++.
5. **Implementar Lógica:** Rellenar los esqueletos con el código real que obtiene la temperatura, estado del disco, etc.
6. **Compilar:** Crear un Subagente (demonio separado que habla con `snmpd` mediante protocolo AgentX) o un módulo dinámico `.so`.
7. **Desplegar:** Ejecutar el subagente y verificar con `snmpwalk`.

